LLM Modules in the Reinforcement Learning Loop: Roles, Architectures, and Credit Assignment

Kourosh Shahnazari[1*] and Seyed Moein Ayyoubzadeh[1]

[1]Sharif University of Technology, Tehran, Iran.

Abstract

Reinforcement learning is increasingly deployed in domains where reward feedback is sparse, delayed, and entangled with long-horizon constraints, making reliable credit assignment difficult. A central development in recent work is the insertion of large language model modules directly into the reinforcement learning loop, not as peripheral interfaces but as components that alter trajectory generation and supervision. In these systems, language modules provide planning priors, structured reward shaping, process verification, synthetic world traces, and tool-memory context that reconfigure optimization at trajectory level. This survey develops a mechanism-first synthesis of that shift. We formalize intervention operators for planning, reward and verifier channels, world construction, and tool-memory mediation; analyze how each operator changes update targets, bias pathways, and stability conditions; and organize the field into a unified taxonomy grounded in optimization effects rather than model branding. We then examine evaluation practice across embodied control, web interaction, games, continuous control, and multi-agent settings, highlighting reproducibility gaps and protocol confounds. Finally, we synthesize recurring failure modes and propose a concrete research agenda on calibration, module authority arbitration, uncertainty-aware simulation, and benchmark design. The resulting perspective positions LLM-in-the-loop reinforcement learning as a systems and optimization discipline centered on trustworthy credit assignment under heterogeneous supervision.

Keywords: large language models, reinforcement learning, planning priors, reward shaping, verification, credit assignment

Index Terms—large language models, reinforcement learning, modular control, trajectory optimization, verification channels, tool-memory mediation.

1 Introduction

1.1 Motivation

Reinforcement learning has produced strong results in narrow, well-shaped environments, but its practical limits become visible when decision-making moves into open-ended settings with delayed feedback, combinatorial action structure, and partial observability. In robotics, a manipulation sequence may succeed or fail only after a long chain of intermediate contacts and safety-sensitive motions. In web interaction, success often requires stateful navigation through dynamically changing interfaces where many early actions appear locally plausible but are globally harmful. In strategic games and text-rich interactive domains, reward sparsity and horizon length make gradient signals weak precisely where policy differentiation is most needed. Under these conditions, optimization fails less because of policy parameterization and more because supervision arrives in forms that are too coarse, too delayed, or too ambiguous to support stable credit allocation.

This weakness is magnified when the environment objective is underspecified. Reward design in realistic tasks is frequently an engineering bottleneck: hand-crafted reward terms can be fragile, easy to exploit, or expensive to maintain under distribution shift. Even when reward is available, the same scalar return can correspond to many qualitatively different trajectories, some robust and some brittle. Standard temporal-difference updates do not automatically encode process quality, causal structure, or semantic constraints. As a result, training often converges toward shortcut behaviors that satisfy proxy feedback while degrading out-of-distribution performance.

Recent systems increasingly address these failures by embedding language modules inside the control loop itself. Instead of treating a language model as an outer wrapper that only generates high-level instructions, these methods route language outputs into the trajectory-generation and update pipeline: subgoal proposals shape exploration, reward and verifier channels densify supervision, synthetic rollouts modify replay, and tool-memory interfaces alter effective state. This is a structural intervention, not a cosmetic one. Once multiple channels participate in update construction, the learning system no longer optimizes solely against raw environment reward; it optimizes against a composite supervision process whose reliability depends on module calibration and arbitration policy [1–6].

A second motivation comes from horizon management. Many long-horizon tasks are difficult because the agent must discover useful intermediate structure before gradients become informative. LLM-driven decomposition can expose semantically coherent latent plans, reducing effective horizon and focusing exploration on causally relevant regions of state-action space. The effect is especially visible in hierarchical control settings where high-level plans, option proposals, or code-like sketches provide scaffolds that low-level RL can refine through grounded interaction [7–10].

A third motivation is state construction under partial observability. In many interactive tasks, useful state is distributed across logs, tools, documents, and previous reasoning traces. Tool-memory modules can integrate these sources into a persistent context that changes both action selection and the meaning of subsequent rewards.

This creates opportunities for better long-term consistency but also introduces new failure surfaces when stale or erroneous context propagates across decisions [11–14].

Viewed at system level, these developments represent a shift from monolithic policy learning toward modular optimization pipelines. In monolithic designs, supervision quality is mostly determined by reward design and value estimation. In modular designs, supervision quality is co-determined by planner priors, verifier calibration, simulation fidelity, and state-construction reliability. The resulting system can be far more capable, but only if these channels are coordinated. This is why many recent gains are accompanied by new classes of failure that do not appear in classical RL benchmarks, including verifier-target mismatch, synthetic replay drift, and tool-context contamination.

This survey argues that the right conceptual unit is not the model, but the intervention pathway. A planner is useful not because it is linguistic, but because it changes which trajectories are explored. A verifier is useful not because it can critique text, but because it changes which updates are trusted. A world module is useful not because it can narrate dynamics, but because it changes replay composition and planning rollouts. Framing the field this way allows architectural comparisons across domains and avoids category confusion between language capability and optimization function.

The same pathway view also clarifies why optimistic claims often fail to transfer across task families. A method that improves robotic manipulation through decomposition may underperform in web agents if tool contracts are unstable. A verifier channel that improves textual process quality may destabilize low-level control if score semantics are weakly correlated with environment return. These outcomes are not contradictions; they reflect different pathway sensitivities. Interpreting results through intervention pathways therefore improves external validity by separating what is module-specific from what is mechanism-general.

A final motivation is methodological maturity. As the literature grows, architectural names proliferate faster than shared design principles. Without a unifying lens, neighboring methods are compared as if they were fundamentally different even when they manipulate the same update pathway. Conversely, methods with similar reported metrics may rely on incompatible intervention contracts. The field now benefits more from synthesis of pathway-level assumptions and authority policies than from additional ad hoc module variants.

1.2 Thesis and Scope Boundary

The thesis of this survey is that LLM-enhanced reinforcement learning is best understood as trajectory-level optimization under heterogeneous supervision. The relevant question is not whether language models are powerful in general, but how specific language-mediated operators intervene in sampling, weighting, and update construction inside the RL loop. This perspective unifies apparently diverse systems and clarifies why similar gains and failure modes appear across robotics, web agents, game-playing, and offline regimes.

Scope is directional. We survey methods where language modules improve reinforcement learning systems by influencing policy learning, planning, reward shaping,

verification, simulation, or state mediation in environment-grounded tasks. Work centered on using RL to improve language models themselves is adjacent context but not the object of analysis here. This boundary matters because the optimization target differs fundamentally: in-scope work aims to improve decision quality and learning dynamics of RL agents; out-of-scope work primarily targets linguistic behavior of the language model.

This directional criterion also avoids conceptual conflation. A method can involve both RL and LLMs while still being irrelevant to RL-system improvement if the environment-grounded control objective is absent. Conversely, a method can remain in scope even when language modules are not the final action policy, as long as they alter trajectory priors, supervision density, or state construction in ways that measurably improve RL performance.

This boundary is especially important for credit-assignment analysis. When the optimized object is an RL agent interacting with an environment, language channels act as supervision modifiers whose quality can be evaluated against grounded returns. When the optimized object is the language model itself, those same channels serve a different objective and different failure modes dominate. Conflating these settings would blur causal interpretation of reported gains. By keeping scope directional, we maintain a coherent optimization unit throughout the survey.

1.3 Contributions of This Survey

This survey makes five contributions. First, it formalizes a unifying framework in which language modules are intervention operators that act on trajectory generation and update pathways. Second, it proposes a mechanism-first taxonomy organized by optimization effects: trajectory prior shaping, supervision restructuring, world construction, search hybrids, data augmentation, and tool-memory mediation. Third, it develops a credit-assignment analysis that distinguishes beneficial densification from bias injection and emphasizes the role of module authority in controlling that trade-off. Fourth, it synthesizes evaluation practice across domain families and identifies protocol-level confounds that limit comparability. Fifth, it consolidates recurring failure modes and articulates a concrete research agenda aimed at calibrated verification, uncertainty-aware simulation, authority arbitration, and reproducible benchmark design.

1.4 Organization of the Paper

Section 2 introduces the minimal RL and LLM preliminaries needed for subsequent analysis. Section 3 presents the unifying framework, formal intervention operators, and core equations for mixed supervision. Section 4 develops the mechanism-first taxonomy with category-level synthesis, architectural patterns, and representative evidence. Section 5 discusses evaluation ecosystems and reporting standards. Section 6 analyzes system-level failure modes and trade-offs. Section 7 outlines ten open research problems. Section 8 concludes with a synthesis of implications for dependable long-horizon RL.

2 Preliminaries

2.1 RL Objective and Trajectory View

We consider a Markov decision process $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ with policy $\pi_\theta(a \mid s)$. The optimization objective is

$$J(\pi) = \mathbb{E}_\tau \left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t) \right], \tag{1}$$

where τ is a trajectory sampled under the policy and environment dynamics. Equation (1) emphasizes that learning quality depends jointly on trajectory distribution and reward informativeness. In long-horizon tasks, two policies can produce similar short-term rewards while diverging sharply in delayed outcomes; thus trajectory structure, not only instantaneous action accuracy, governs final performance.

A trajectory is written as

$$\tau = (s_0, a_0, r_0, \ldots, s_T). \tag{2}$$

Equation (2) is central because every LLM intervention considered in this survey ultimately modifies one of its components: state representation, action proposal, reward channel, or replay composition. Treating LLM modules as trajectory operators clarifies when gains derive from true control improvement versus superficial proxy optimization.

For policy-gradient methods, a compact expression is

$$\nabla_\theta J(\pi_\theta) = \mathbb{E} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \, \hat{A}_t \right]. \tag{3}$$

Equation (3) highlights where credit assignment enters: the advantage estimate $\hat{A}_t$ determines how strongly each action affects updates. Sparse or delayed rewards make $\hat{A}_t$ noisy and high-variance, which motivates auxiliary supervision channels that can densify learning signals. However, any auxiliary channel can also introduce systematic bias if it is misaligned with environment outcomes.

2.2 Credit Assignment Pathways

In conventional RL, supervision primarily flows from environment reward through returns, value targets, or temporal-difference errors. This pathway is grounded but often weak in open-ended tasks. Reward shaping and auxiliary losses are therefore common, but their reliability depends on how faithfully they track task success over long horizons.

The key challenge is temporal and structural ambiguity. Long delays obscure causal contribution of early actions; dense interaction logs may include many irrelevant steps; and partial observability means that critical state information may never be directly encoded in raw observations. Credit assignment therefore becomes a systems problem:

what information channels are trusted, when they are trusted, and how their influence is weighted during updates.

Offline RL and model-based RL expose complementary versions of this issue. Offline methods inherit dataset support constraints; new supervision channels can help by relabeling or augmenting data, but can also worsen extrapolation error if synthetic samples are unrealistic. Model-based methods can improve sample efficiency through imagined rollouts, but model bias can dominate when simulated dynamics drift from real transitions [15–18].

From an estimation viewpoint, each additional supervision pathway perturbs both bias and variance. Dense shaping often reduces variance by supplying frequent learning signals, yet it can increase bias if the shaping function tracks stylistic regularities rather than causal progress. Verifier gating can reduce variance by rejecting noisy trajectories, but can increase bias when calibrated confidence diverges from true task utility. Offline augmentation can reduce variance in low-support regions, but can amplify bias if synthetic traces systematically miss environment constraints. This recurring bias-variance exchange is a central thread across all taxonomy categories discussed later.

2.3 What LLM Modules Add in Decision Settings

Language modules contribute three capabilities that are directly relevant to RL optimization. First, they provide structured priors over latent plans, subgoals, and action abstractions, which can reduce search complexity. Second, they can transform weak scalar feedback into denser process supervision by generating reward terms, critiques, or verifier scores. Third, they can mediate state by integrating tool outputs and memory traces into contextual summaries used for action selection and update filtering.

These capabilities are useful because they target the same bottlenecks that cause instability in long-horizon RL: sparse supervision, weak decomposition, and incomplete state. Yet usefulness is conditional. Any module output enters learning as a model-dependent signal with uncertainty and potential bias. High performance therefore requires explicit arbitration between grounded environment feedback and language-derived channels rather than unconditional trust in generated content [19–22].

From this perspective, "LLM for RL" is not a single method class but a family of control interventions with different statistical properties. Planner channels primarily affect exploration and support. Reward and verifier channels primarily affect target construction. Simulation channels primarily affect data distribution. Tool-memory channels primarily affect observability. Each pathway introduces a distinct bias-variance profile, and practical systems must tune these profiles jointly. This motivates the mechanism-first taxonomy adopted later in the paper.

Another implication is that evaluation should move beyond aggregate return reporting. Two systems can achieve similar return with very different supervision structures and thus very different robustness properties. A planner-heavy method may fail under decomposition shift, while a verifier-heavy method may fail under reward-proxy exploitation. Without pathway-specific diagnostics, these differences remain hidden until deployment.

The broader methodological consequence is that experiment design should align with mechanism claims. If a paper claims improvement through planning priors, it should report decomposition robustness and feasibility diagnostics. If it claims improvement through verifier channels, it should report calibration and anti-gaming evidence. If it claims gains through world construction, it should quantify synthetic-real divergence and uncertainty control. This alignment between claimed mechanism and reported evidence is still inconsistent in the literature, and strengthening it would materially improve interpretability and reproducibility.

The same logic motivates stronger reporting of intervention contracts. A contract specifies what a module may output, where that output enters optimization, and how influence is bounded under uncertainty. Contracts make comparisons fairer because they expose whether gains come from model quality, authority level, or hidden implementation details. In their absence, replicability suffers: apparently minor choices such as memory truncation, tool timeout handling, or verifier thresholding can dominate outcomes. A contract-centric perspective therefore complements algorithm-centric benchmarking and supports clearer cumulative progress.

Finally, preliminaries should be read as a map of supervision pathways, not as a separate background chapter detached from method design. The same policy-gradient estimator can behave very differently depending on which pathway contributes to $\hat{A}_t$ and how much authority each pathway receives. This insight motivates the next section, where module interventions are formalized as operators with explicit semantics and bounded authority.

3 Unifying Framework: LLM Modules Embedded in the RL Loop

3.1 Formal Intervention View

Let M_ϕ denote a language module with parameters ϕ and context input h_t that includes state history, tool traces, and optional memory. The module emits an intermediate variable o_t:

$$o_t = M_\phi(h_t). \tag{4}$$

The policy can then condition on augmented state and module outputs. This formalization treats language outputs as control-relevant latent variables rather than free-form text artifacts.

A planner module instantiates this idea through latent subgoal variables:

$$\pi(a_t \mid s_t, z_t), \qquad z_t = \mathrm{Planner}_\phi(h_t). \tag{5}$$

Equation (5) captures hierarchical conditioning. The planner compresses long-horizon structure into shorter-horizon subproblems. Benefits arise when subgoals are feasible and informative; instability arises when generated plans are semantically coherent yet dynamically incompatible with low-level control.

A reward-shaping module modifies immediate training targets:

$$r'_t = r_t + \alpha\, b_t, \tag{6}$$

where b_t is language-derived shaping and α controls authority. Equation (6) can reduce variance and accelerate learning, but only if shaped terms preserve task semantics. If b_t overweights superficial process features, optimization may drift toward proxy behaviors.

Verifier-mediated weighting can be expressed as an expectation over reweighted trajectory loss:

$$\mathbb{E}_\tau \left[w(\tau)\, L(\tau) \right], \tag{7}$$

with $w(\tau)$ derived from verifier scores. Equation (7) clarifies that verifier channels do not only add information; they reshape effective data distribution seen by the optimizer. This mechanism can suppress noisy trajectories, but it can also hide failure modes if the verifier is systematically overconfident.

Offline augmentation introduces synthetic traces:

$$\mathcal{D}' = \mathcal{D} \cup \hat{\mathcal{D}}, \tag{8}$$

where $\hat{\mathcal{D}}$ contains generated or relabeled trajectories. Equation (8) highlights the central trade-off: broader coverage versus distribution shift. Effective systems bound synthetic influence with confidence-aware filters and consistency checks against real rollouts.

Tool-memory mediation can be written as

$$m_{t+1} = U(m_t, \mathrm{tool}(s_t), o_t)\,, \tag{9}$$

where m_t is persistent context, $\mathrm{tool}(s_t)$ denotes external retrieval or API observations, and o_t is module output used for memory update decisions. Equation (9) makes explicit that memory is not passive storage; it is a learned state-construction process. Errors in U can persist across many steps and alter both action selection and subsequent supervision interpretation.

Taken together, Eqs. (5)–(9) describe a family of operators that intervene at different points but couple through shared trajectories. This coupling explains why isolated ablations may underestimate risk: a planner failure can be amplified by a verifier; a memory error can distort planner context; a world-model bias can change verifier inputs. Reliable design therefore depends on joint reasoning across operators, not independent tuning.

3.2 Intervention Operators and Channel Semantics

The framework uses five operator classes. The planner operator emits latent decomposition variables that alter exploration and option selection. The reward operator emits shaping terms that modify immediate targets. The verifier operator emits process scores that gate, rank, or reweight trajectory updates. The world operator emits imagined transitions that affect planning or replay. The tool-memory operator updates contextual state from retrieval and execution traces.

These operators are compositional. A single system can use planner outputs to propose subgoals, verifier scores to select among candidate rollouts, and world-model

traces to train value functions. Compositionality creates expressive power but complicates diagnosis: when performance changes, attribution must separate planner quality, verifier calibration, simulator reliability, and arbitration policy.

Channel semantics matter as much as model architecture. A verifier score interpreted as hard gating behaves differently from the same score used as soft weighting. A world-model trace used for candidate ranking differs from one used as direct replay data. Robust design therefore requires explicit contracts for each channel: what it can change, when it can change it, and how confidence affects authority.

3.3 Credit Assignment Implications

LLM modules reshape credit assignment through three mechanisms. First, they densify supervision by converting delayed outcomes into intermediate signals. Second, they alter trajectory priors through decomposition and search guidance. Third, they modify data distribution through filtering and synthetic augmentation. Each mechanism can improve sample efficiency, but each introduces model-dependent bias.

Densification improves optimization when auxiliary signals preserve causal relevance. If shaping and verifier channels track process quality that predicts long-term return, advantage estimates become more informative and variance drops. If they track easily exploitable proxies, the same channels amplify misalignment. This explains why verifier calibration and disagreement monitoring are recurring best practices [19, 20, 23, 24].

Trajectory-prior interventions improve horizon management when planner proposals respect environment dynamics. In hierarchical settings, language decomposition can reduce combinatorial search and accelerate curriculum progression. Yet planner authority must be bounded by feasibility checks; otherwise policy updates are trained on unattainable intent structures and convergence degrades [6, 25, 26].

Distribution-shaping interventions are especially sensitive. Synthetic trajectories can fill support gaps in offline or expensive environments, but unfiltered generation can create high-confidence error cascades. Practical systems now combine synthetic replay quotas, uncertainty-aware weighting, and periodic re-anchoring to real interaction return [27–29].

An underappreciated implication is that module interventions change effective objective geometry even when nominal reward is unchanged. Reweighting alters which trajectories dominate gradient estimates; shaping alters local smoothness of learning signals; decomposition alters reachable policy neighborhoods under finite compute. These effects can improve optimization conditioning, but they can also create deceptive plateaus where auxiliary channels appear to converge while grounded return stagnates. Diagnosing this gap requires channel-specific monitoring rather than aggregate reward alone.

3.4 Module Authority and Arbitration

A practical way to reason about safety and stability is module authority. Advisor-level modules propose candidates without direct control. Controller-level modules

affect action selection or reward terms. Judge-level modules gate updates or trajectories. Simulator-level modules influence the data regime itself. As authority increases, potential sample-efficiency gains increase, but so does the risk of systematic error.

Arbitration policies determine whether authority is static or adaptive. Static weighting is simple but brittle under shift. Adaptive arbitration uses uncertainty, disagreement, and environment-grounded diagnostics to adjust influence online. Empirically, systems with explicit arbitration tend to degrade more gracefully when tool quality, prompt format, or domain dynamics change.

3.5 Operator Composition in Practice

Real systems rarely deploy a single operator in isolation. Planner outputs influence action priors, verifier outputs influence update trust, and world outputs influence replay composition at the same time. Composition creates leverage because each operator can compensate weaknesses of another: verifiers can reject planner-induced shortcuts; planners can constrain expensive search; world models can pre-screen poor branches before real interaction. Composition also creates coupling risk because correlated errors can align in harmful ways. For example, a planner may generate semantically attractive but infeasible trajectories, while a verifier trained on linguistic coherence assigns them high confidence, and a simulator reproduces the same bias in synthetic replay.

To manage coupling risk, practical pipelines increasingly adopt staged composition. Early training uses conservative authority, with planner guidance and verifier weighting applied softly while environment reward dominates updates. As calibration evidence accumulates, module influence is increased in states where disagreement is low and uncertainty is bounded. During deployment or shift-sensitive phases, influence can be reduced dynamically when diagnostics indicate channel drift. This staged approach is less elegant than fixed-policy integration but has better empirical stability in open-ended tasks.

A second practical pattern is counterfactual channel testing. Instead of only evaluating final policy quality, systems are tested under channel ablation and perturbation: planner disabled, verifier noise injected, synthetic replay removed, or tool outputs delayed. The resulting performance deltas reveal which pathways are essential and which are brittle. Such diagnostics are especially important for multi-module systems where nominal gains may be driven by hidden dependencies that do not transfer across environments.

4 Mechanism-First Taxonomy

4.1 Trajectory Prior Shaping (Planning and Decomposition)

Trajectory prior shaping methods use language modules to constrain search before low-level action optimization begins. The core intuition is that long-horizon RL is often difficult because unstructured exploration wastes samples on semantically irrelevant branches. Planner modules address this by proposing subgoals, option candidates, or executable sketches that induce a more informative trajectory prior. In practice, these

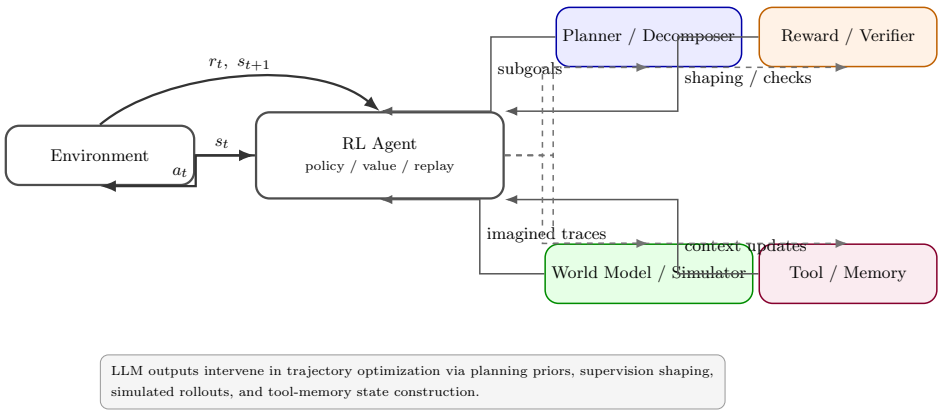

Fig. 1 LLM modules embedded in the reinforcement learning loop. Planner, reward-verifier, world-model, and tool-memory modules intervene in trajectory generation and update construction while environment reward remains the grounding anchor.

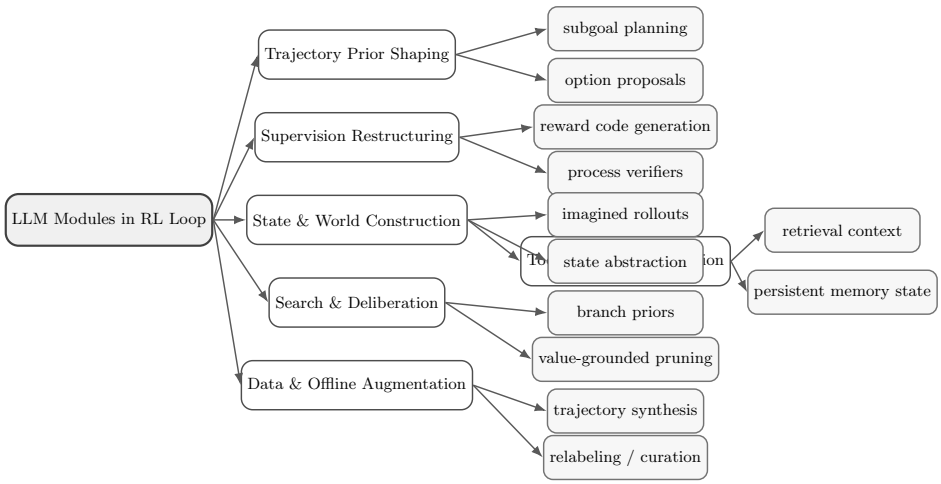

Fig. 2 Taxonomy schematic organized by optimization intervention pathway.

systems instantiate a two-level interface: language-guided high-level planning and RL-grounded low-level execution. This structure can compress effective horizon, improve curriculum smoothness, and produce better early-stage sample efficiency. The benefit is strongest when the environment exhibits compositional substructure, such as manipulation pipelines, web workflows, and multi-step game objectives. Planner-guided systems also improve interpretability because high-level intent is exposed as explicit latent variables. At the same time, explicit plans create a new coupling problem: if planner semantics drift from control feasibility, low-level updates inherit conflicting supervision.

A dominant design pattern is latent subgoal conditioning, where planner outputs parameterize option policies or behavior priors. Some methods keep planner influence

Category	Intervention point	Typical signals / outputs	Typical RL setting	Rep. refs
Trajectory Prior Shaping	Action selection and option interface	Subgoals, plan sketches, option proposals	Hierarchical RL, long-horizon control	[1, 6]
Supervision Restructuring	Reward and update target channels	Reward code, critique, verifier scores	Sparse-reward online RL, process-supervised RL	[4, 20]
State & World Construction	Model/replay path and latent state	Imagined rollouts, synthetic transitions, abstractions	Model-based RL, expensive interaction regimes	[15, 16]
Search & Deliberation Hybrids	Planning-time branching and pruning	Branch priors, candidate rationales, scoring	Tree-search control, strategic tasks	[30, 31]
Data & Offline Augmentation	Dataset and replay composition	Synthetic demos, relabeling, trajectory imputation	Offline RL, low-interaction settings	[27, 32]
Tool & Memory Mediation	State construction under partial observability	Retrieval context, API outputs, persistent memory	Web agents, long-context embodied interaction	[5, 11]
Multi-agent / Interactive	Coordination and social credit pathways	Shared plans, negotiation feedback, role traces	Team RL and interactive planning	[33, 34]

Table 1 T1. Mechanism-first taxonomy of LLM modules in the RL loop.

soft, using proposals as candidate sets ranked by value estimates; others treat plans as mandatory scaffolds and train controllers to imitate or refine them. Soft integration tends to be more robust under shift but may underutilize planner structure. Hard integration gives stronger horizon compression but can fail abruptly when generated subgoals are inconsistent with dynamics [1, 8, 10, 25].

A second pattern is code- or program-mediated decomposition, where the language module emits executable control templates. RL then optimizes parameters, branching choices, or execution-time repair around that template. This can reduce search dimensionality and encode domain heuristics efficiently. However, it increases sensitivity to interface design: if program outputs are underconstrained, syntax-level validity may hide semantic failure; if overconstrained, adaptation capacity is lost. Effective systems include typed action schemas, feasibility critics, and rapid plan-repair loops [2, 7, 35, 36].

Evaluation in this category typically uses long-horizon success metrics, option completion rates, and sample-efficiency curves under sparse rewards. Robotics and embodied benchmarks test plan feasibility under physical constraints, while web and interactive tasks test decomposition quality under stochastic interfaces. Reported

gains are often largest in early training and in tasks where subgoal structure is human-interpretable.

The primary failure mode is planner-policy mismatch. A planner may propose semantically coherent but physically infeasible steps, causing persistent recovery loops. Another failure is decomposition overfitting, where fixed plan templates succeed on benchmark patterns but fail on slight distribution shifts. Mitigations include uncertainty-aware planner confidence, controller-side rejection of infeasible subgoals, and periodic planner retraining from grounded execution traces [6, 9, 26].

Takeaway: trajectory prior shaping is one of the most effective ways to reduce horizon difficulty, but only when planner authority is calibrated by environment-grounded feasibility checks. Planner quality alone is insufficient; stability depends on planner-controller contracts and repair policies.

4.2 Supervision Restructuring (Reward Design and Verifiers)

Supervision restructuring methods intervene directly in update targets by producing reward terms, process critiques, or verifier scores. The central idea is to replace sparse scalar outcomes with denser supervision that reflects progress quality along the trajectory. This is attractive in tasks where final success appears late and intermediate behavior quality matters. Reward-generation pipelines translate task descriptions into shaped objectives; verifier pipelines score trajectory quality and use scores for gating or weighting updates. In both cases, the language module modifies effective optimization targets, not merely interpretation layers.

One design family generates explicit reward functions or code snippets from textual task definitions. These methods reduce manual reward engineering and can quickly adapt objectives across tasks. The challenge is proxy fidelity: generated rewards may privilege stylistic regularities or shallow heuristics that correlate weakly with final success. Robust implementations therefore pair reward generation with runtime auditing, constraint checks, and ablation against pure environment reward baselines [4, 37–39].

A second family emphasizes verifiers and process critics. Instead of directly replacing reward, verifier outputs influence trajectory selection, update weighting, or replay prioritization. This can improve robustness in noisy domains by suppressing clearly flawed traces. Yet verifier authority is dangerous when calibration is weak: agents may optimize verifier shortcuts and appear to improve while true task competence stagnates. Recent systems mitigate this via disagreement-aware ensembles, periodic grounding checks, and conservative weighting schedules that increase verifier influence only after calibration diagnostics stabilize [19–21, 23, 24].

Evaluation in this category should report more than final return. Useful diagnostics include verifier-environment correlation, false-positive acceptance rates, and sensitivity of policy gains to verifier removal. Domains include robotics reward coding, web process supervision, and strategic tasks where intermediate reasoning quality predicts delayed outcomes.

Failure modes include reward hacking, verifier exploitation, and authority inversion where auxiliary channels overpower environment reward. Mitigations include

bounded shaping coefficients, dual-objective training with explicit grounding constraints, calibration stress tests under adversarial perturbations, and counterfactual replay audits.

Takeaway: supervision restructuring can transform sparse-reward RL, but gains are trustworthy only when verifier and shaping channels remain explicitly tethered to grounded outcomes and are monitored as potential bias sources.

4.3 State and World Construction (World Models, Simulation, Imagination)

State and world construction methods use language modules to build representations of environment dynamics, latent task state, or imagined trajectories. Their goal is to improve sample efficiency and planning quality when real interaction is expensive. In model-based variants, language modules approximate transition structure or provide high-level simulation priors. In replay-centric variants, they synthesize candidate trajectories for value pretraining, policy warm-start, or uncertainty probing.

A common pattern is hybrid imagination: synthetic rollouts are generated then filtered by consistency checks before entering replay. This preserves efficiency gains while limiting model bias. Another pattern uses language modules to produce abstract scenario descriptions that guide model-based planning rather than directly replacing dynamics. The latter is safer when low-level dynamics are difficult for language models to capture precisely [15, 16, 18, 29].

Uncertainty modeling is decisive. Without confidence estimates, synthetic traces can dominate replay and induce value drift. Strong systems use uncertainty-aware weighting, capped synthetic-real ratios, and divergence monitoring between imagined and observed transitions. Some methods also separate planning simulation from policy training replay, reducing contamination of core value targets [17, 40–42].

Evaluation emphasizes sample efficiency under constrained interaction, transfer to real environments, and robustness under dynamic shift. Embodied and web domains test whether imagined traces remain useful when environment details change. Offline settings test whether world priors improve generalization without amplifying support mismatch.

Failure modes include hallucinated dynamics, overconfident synthetic replay, and representational shortcuts where language abstractions collapse physically relevant distinctions. Mitigations include conservative synthetic budgets, uncertainty calibration, and recurrent grounding checkpoints with real rollouts.

Takeaway: world construction can substantially improve efficiency, but it is structurally high-risk because it modifies the data regime itself. Reliability depends on disciplined uncertainty management and tight grounding to real transitions.

4.4 Search and Deliberation Hybrids

Search and deliberation hybrids integrate language priors into branching and pruning while retaining RL or value-based grounding. The motivation is that long-horizon decision quality can improve when planning-time computation is allocated adaptively rather than uniformly. Language modules propose candidate branches, rationale

traces, or decomposition hints; RL signals evaluate candidates under environment-grounded objectives.

One architectural pattern uses language proposals to narrow branching factor before value scoring. This reduces combinatorial explosion in large action spaces and can improve planning efficiency. Another pattern performs iterative self-search, where the system alternates proposal, critique, and value evaluation loops. The latter often improves quality but has higher latency and orchestration complexity [30, 31, 43, 44].

A key design decision is budget scheduling. Fixed-depth deliberation is predictable but inefficient under variable task difficulty. Adaptive schemes tie search depth to confidence, uncertainty, or verifier disagreement. This can preserve responsiveness while maintaining quality on hard instances. However, adaptive policies require reliable confidence estimation; otherwise they misallocate compute and amplify instability.

Evaluation typically reports quality-latency trade-offs, branch efficiency, and robustness under perturbations. In interactive domains, response-time constraints matter as much as final success. Strategic and coding-oriented tasks often show large gains from deliberation, but portability to embodied control remains limited when planning latency is tightly constrained.

Failure modes include over-deliberation, branch prior collapse, and mismatch between linguistic plausibility and environment feasibility. Mitigations include capped deliberation budgets, diversity-promoting branch sampling, and explicit environment-grounded rejection criteria.

Takeaway: search hybrids are powerful for hard reasoning trajectories, but their value depends on disciplined compute allocation and strict grounding of language priors.

4.5 Data and Offline Augmentation

Data and offline augmentation methods intervene on training support by generating demonstrations, relabeling transitions, and synthesizing trajectories. This family is most relevant when online interaction is costly, risky, or unavailable. The central promise is to increase data coverage in regions that matter for policy improvement while preserving conservative learning under support constraints.

A common strategy is trajectory relabeling with semantic annotations or latent goal reinterpretation. Language modules can infer intent structure from raw traces, enabling richer supervision for value and policy learning. Another strategy synthesizes demonstrations from textual task descriptions or prior logs, then filters them through consistency criteria before integration. Both approaches can improve initial policy quality and reduce exploration burden [27, 28, 32, 45].

The dominant challenge is confounding through distribution shift. Synthetic traces may be internally coherent yet misaligned with environment dynamics or reward semantics. Over-reliance can produce deceptive offline gains that fail during deployment. Effective pipelines enforce provenance tracking, sample-level confidence scores, and conservative mixing schedules that gradually increase synthetic influence only when validation improves.

Evaluation should include support-aware metrics, out-of-distribution performance, and robustness to synthetic-noise injection. Pure aggregate return is insufficient

because it hides whether gains depend on fragile synthetic artifacts. Benchmarks with held-out dynamics and adversarial perturbations are especially informative for this category.

Failure modes include dataset contamination, confirmation bias from self-generated traces, and hidden leakage of benchmark patterns through generation prompts. Mitigations include strict split hygiene, independent verification filters, and stress tests that isolate dependence on synthetic subsets.

Takeaway: offline augmentation is a high-leverage path for data-limited RL, but only when synthetic data governance is treated as a first-class optimization and safety problem.

4.6 Tool and Memory Mediation

Tool and memory mediation methods treat external tools, retrieval systems, and persistent memory as part of effective state. This is particularly important in partially observable settings where critical information is distributed across interaction history, APIs, logs, or external knowledge sources. By constructing richer latent state, these methods can improve long-horizon consistency and recovery behavior.

Architecturally, systems often include a retrieval layer, a memory update policy, and a controller that conditions on both observation and memory. Language modules help normalize heterogeneous tool outputs into decision-ready context. In some methods, memory itself is optimized through RL signals, creating a closed loop between context management and policy quality [5, 11–13].

A persistent issue is error accumulation. Incorrect tool calls or stale memory entries can propagate over many steps, producing delayed and difficult-to-diagnose failures. Strong systems use typed tool interfaces, schema validation, temporal decay, and rollback mechanisms. Some methods maintain parallel memory hypotheses and select among them via verifier scores to reduce lock-in.

Evaluation focuses on multi-turn reliability, recovery after incorrect intermediate actions, and robustness to missing or noisy tools. Web and interactive domains are natural testbeds because tool contracts can vary at runtime. Embodied domains increasingly adopt memory mediation for long task sequences with intermittent observability.

Failure modes include tool hallucination, hidden-state leakage, and latency spikes due to excessive retrieval. Mitigations include confidence-aware query gating, bounded memory growth, and explicit penalties on unnecessary tool calls.

Takeaway: tool-memory mediation reframes RL state construction as a controllable subsystem. When calibrated, it improves robustness under partial observability; when unmanaged, it becomes a persistent error amplifier.

4.7 Multi-agent and Interactive Settings

Multi-agent settings expose additional credit pathways because outcomes depend on coordination quality and communication structure. Language modules can provide shared plan abstractions, role assignment proposals, and social-feedback verifiers that

shape team learning. These interventions are promising for tasks where collective strategy evolves over long horizons.

Design patterns include centralized planning with decentralized execution, peer-verification loops, and role-conditioned memory sharing. A key challenge is non-stationarity: each agent's adaptation changes the data distribution faced by others. Language-mediated coordination can reduce this non-stationarity by stabilizing high-level intent, but only when communication channels are reliable and resistant to shortcut signaling [33, 34, 46, 47].

Evaluation should report both individual and team-level metrics, communication efficiency, and robustness to partial communication failure. Failure modes include emergent collusion on proxy signals, communication drift, and instability from asynchronous authority updates. Mitigations include communication regularization, role-specific verifier constraints, and staged authority schedules.

Takeaway: interactive settings make module arbitration more complex but also more revealing. They provide stress tests for whether language-mediated credit signals remain reliable under coupled adaptation.

4.8 Cross-Category Interaction Patterns

Although the taxonomy separates mechanisms for clarity, the most capable systems are typically hybrids that combine at least three categories. A common pattern in long-horizon settings is planner plus verifier plus tool-memory. The planner proposes coarse structure, the tool-memory channel stabilizes state under partial observability, and the verifier filters process errors before updates. This combination often outperforms any individual component because it jointly addresses horizon length, observability gaps, and supervision quality. The same combination can also fail catastrophically if authority scheduling is misconfigured, showing why interaction analysis is as important as module design.

Another recurrent pattern is world construction plus offline augmentation. Synthetic rollouts are used to extend support, while relabeling or trajectory synthesis refines data semantics. This can accelerate learning in low-interaction regimes, but only if generated traces are uncertainty-filtered and proportionally mixed with real data. Over-reliance on generated data produces deceptive improvements in closed benchmarks and brittle behavior under deployment shifts.

Search hybrids often act as a bridge between planning and verification. Language priors narrow candidate branches; value or verifier channels rank candidates; environment feedback closes the loop. When these stages are balanced, deliberation quality rises without prohibitive latency. When balance fails, systems either over-deliberate on weak branches or collapse to heuristic priors that ignore grounded correction.

Across categories, one design principle repeatedly appears: module influence should be conditional on evidence quality. Conditional influence can be implemented by confidence thresholds, disagreement gating, or state-dependent arbitration policies. Unconditional influence, even from strong modules, tends to reduce robustness because error rates vary across states and phases of training.

A second principle is pathway observability. Operators should expose diagnostics that map directly to failure hypotheses: planner feasibility rates, verifier calibration

Signal channel	Where it enters	Benefit	Bias risk	Diagnostics	Rep. refs
Planner subgoals	Action prior and option routing	Horizon reduction and focused exploration	Infeasible decomposition	Plan-feasibility and repair rate	[1, 6]
Reward shaping	Immediate target modification	Denser supervision under sparse reward	Proxy reward exploitation	Shaping-return correlation, ablation delta	[4, 37]
Verifier weighting	Update gating and replay weighting	Noise suppression, process control	Shortcut optimization of verifier	Calibration curves, disagreement rate	[19, 20]
Synthetic rollouts	Replay and model-based planning	Sample-efficiency gains	Hallucinated dynamics	Synthetic-real divergence, uncertainty score	[16, 17]
Tool-memory state	Observation augmentation	Better partial-observation handling	Error persistence across turns	Memory freshness, rollback frequency	[5, 12]

Table 2 T2. Optimization and credit-assignment channels induced by LLM modules.

curves, synthetic-real divergence, tool reliability traces, and authority transition logs. Without such diagnostics, post hoc debugging becomes speculative and often conflates root causes.

A third principle is phase-aware optimization. During early exploration, planner and world channels can provide substantial gains by reducing uninformative search. During late-stage refinement, aggressive auxiliary influence may become counterproductive if it dominates grounded reward. Successful systems therefore shift emphasis over time, moving from guidance-heavy to grounding-heavy updates as task competence increases.

Cross-category synthesis also suggests a useful decomposition of reliability: semantic reliability (are generated suggestions meaningful), dynamical reliability (are they feasible in environment dynamics), and supervisory reliability (do they improve update targets). A module can score well on one axis and poorly on others. For instance, richly articulated plans may be semantically strong but dynamically weak; strict verifiers may be supervisory strong in-distribution but brittle under shift. Reporting these axes separately would make category comparisons more informative and help prevent overgeneralization from narrow benchmark gains.

An additional synthesis point is interoperability cost. Categories that are individually effective can still conflict when composed if their interfaces are under-specified. Planner tokens, verifier labels, and memory schemas may encode incompatible assumptions about time scale, causality, or state abstraction. Explicit interoperability design, including typed interfaces and shared uncertainty semantics, is therefore an overlooked determinant of system reliability and should be treated as a first-class architectural objective.

5 Evaluation and Benchmarking Ecosystem

Evaluation for LLM-in-the-loop RL remains fragmented because methods differ simultaneously in task domains, intervention channels, and reporting granularity. A planner-heavy method in embodied manipulation, a verifier-heavy method in web interaction, and a simulation-heavy method in offline RL may all report "success rate" while measuring substantially different phenomena. As a result, direct comparison without channel-aware context can be misleading.

A robust evaluation perspective separates outcome metrics from pathway diagnostics. Outcome metrics include final return, success rate, constraint satisfaction, and latency-aware utility. Pathway diagnostics include planner feasibility, verifier calibration, synthetic-real divergence, tool reliability, and arbitration behavior. Without pathway diagnostics, improvements may be attributed to the wrong mechanism, and replication becomes unreliable.

Domain families introduce different confounds. Robotics and embodied control emphasize safety and dynamics realism; simulated gains may not transfer if world priors are overfit. Web and interactive tasks emphasize stateful recovery under changing interfaces and tool noise. Games and strategic settings emphasize long-horizon reasoning and branch management. Offline and batch settings emphasize support mismatch and extrapolation risk under synthetic augmentation. Multi-agent settings add non-stationarity and coordination-credit ambiguity.

Fragmentation is also protocol-level. Prompt templates, context-window management, tool permission policies, memory decay rules, and verifier thresholds often function as hidden hyperparameters. Two studies with similar architectures can diverge materially because these contracts are undocumented or tuned differently. A mature evaluation standard should therefore report intervention contracts explicitly, including how module authority is scheduled and how disagreements are resolved.

Another weakness is insufficient stress testing under shift. Many studies evaluate on in-distribution benchmarks where language priors align with task phrasing. Robustness claims require perturbation protocols: prompt paraphrase, tool outages, delayed reward perturbations, and adversarial verifier cases. Systems that maintain performance under these shifts are more likely to generalize beyond curated benchmarks.

A stronger benchmarking practice is to pair performance metrics with pathway stress maps. A stress map reports which channels fail first under controlled perturbations and how rapidly recovery occurs after arbitration adjustments. For planner-heavy methods, this may involve decomposition perturbation and feasibility degradation curves. For verifier-heavy methods, this involves calibration drift under paraphrase and adversarial process traces. For world-construction methods, it involves synthetic-real divergence under dynamics shift. Such maps convert robustness from a generic claim into a measurable property of intervention pathways.

Recommended reporting should include five elements. First, explicit intervention taxonomy labels for each module used. Second, authority schedules describing when and how each module can alter updates. Third, calibration diagnostics for verifier and shaping channels. Fourth, synthetic-replay accounting and uncertainty filters where

Domain family	Common tasks	Metrics	Pitfalls	Recommended reporting	Rep. refs
Embodied / Robotics	Manipulation, navigation, long-horizon assembly	Success, return, safety constraints	Simulator-to-real mismatch, brittle reward specifications	Report constraint violations, plan feasibility, intervention ablations	[1, 16]
Web / Interactive	Multi-step browsing and tool workflows	Task success, step cost, recovery latency	Tool availability variance, context drift	Report tool contract, memory policy, recovery metrics	[5, 14]
Games / Text / Strategy	Long-horizon sparse-reward control	Return, win rate, horizon completion	Prompt leakage, over-deliberation	Report branching budget and verifier diagnostics	[3, 30]
Offline / Batch RL	Policy learning from fixed datasets	Normalized return, OOD robustness	Synthetic-data confounding, support mismatch	Report synthetic-real ratio and shift diagnostics	[27, 32]
Multi-agent	Coordination, negotiation, role allocation	Team reward, coordination success	Non-stationarity, proxy collusion	Report communication protocol and role stability	[33, 34]

Table 3 T3. Benchmark families, protocol pitfalls, and recommended reporting checklist.

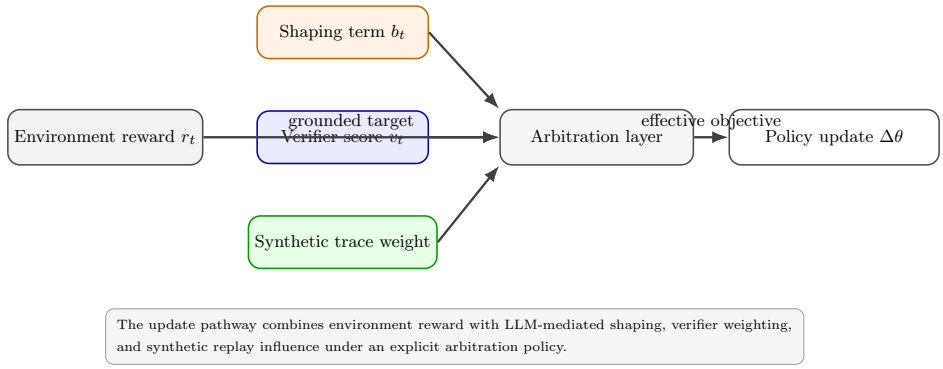

Fig. 3 Supervision pathways in LLM-augmented RL. Environment reward, shaped reward, verifier weighting, and synthetic replay are arbitrated before policy updates.

world or data generation is used. Fifth, stress-test protocols under perturbation of prompts, tools, and environment dynamics.

Evaluation should also separate capability from dependency. A method that performs well only with fragile prompt templates or unrestricted tools may offer limited practical value. Reporting dependence curves - performance versus tool reliability, memory truncation, or verifier noise - can reveal whether gains are structural or brittle.

One practical recommendation is to report channel attribution studies alongside main benchmarks. Attribution can be done with intervention ablations, confidence perturbations, and selective channel freezing. The objective is not to prove causal certainty in every setting, but to prevent hidden dependencies from being misinterpreted as general algorithmic improvements. For example, a large gain that disappears when verifier influence is bounded indicates a calibration-dependent effect, not a universally robust policy improvement.

A second recommendation is to include stress suites that target each pathway explicitly: decomposition shift for planners, adversarial paraphrase for verifiers, simulator perturbation for world modules, and tool outage scenarios for memory-mediated agents. Such suites make reported gains more informative because they reveal where methods fail and how fast they recover.

A third recommendation concerns reproducibility metadata. Studies should document context limits, tool permissions, memory update policies, arbitration thresholds, and synthetic replay budgets. These details are frequently omitted despite acting as decisive hyperparameters. Transparent reporting improves cross-paper comparability and reduces replication cost for the community.

Finally, evaluation should distinguish short-horizon benchmark competence from sustained trajectory quality. Many failures emerge only in long interactions where small errors compound. Reporting long-horizon consistency, recovery latency, and degradation under perturbation provides a more realistic estimate of deployment readiness.

Beyond protocol design, evaluation should reflect deployment constraints. In many settings, compute and latency budgets are as important as asymptotic return. A method that requires expensive deliberation may outperform in offline scoring but underperform in real-time control. Reporting quality-latency frontiers and channel activation cost profiles helps identify which systems are viable under practical constraints. This is particularly important for interactive environments where delayed actions degrade task outcomes regardless of policy quality.

Another important consideration is transfer granularity. Cross-domain transfer should not be reported only as aggregate success, but as pathway transfer: does planner quality transfer, does verifier calibration transfer, does memory policy transfer? Distinguishing these components reveals whether a method generalizes by learning robust control principles or by inheriting domain-specific scaffolding. Such decomposition can guide future work toward reusable mechanisms rather than brittle task-specific orchestration.

In short, credible evaluation in this area requires dual accountability: capability accountability (does return improve) and pathway accountability (why it improves, and when it fails). This dual lens improves comparability across domains and prevents over-interpretation of gains driven by fragile intervention contracts.

It also enables cumulative science: when pathway evidence is reported consistently, subsequent work can reuse diagnostics, isolate genuinely new contributions, and avoid repeating the same hidden confounds under different naming conventions.

6 Failure Modes and System Trade-offs

LLM modules improve RL by adding structure, but they also introduce new coupled failure mechanisms. The most recurrent failures are not isolated model mistakes; they are pathway conflicts between planner intent, verifier incentives, simulation assumptions, and tool-mediated state. Understanding these failures requires analyzing interaction topology, not only module accuracy.

6.1 Planner-policy mismatch

Planner-policy mismatch occurs when high-level decomposition quality and low-level executability diverge. The planner can be semantically correct while the controller lacks the skill or context to realize proposed subgoals. Repeated mismatch leads to wasted rollouts, unstable credit assignment, and superficial compliance behaviors. Mitigations include dynamic plan repair, feasibility critics, and confidence-aware fallback from planner-conditioned control to baseline policies.

The subtlety is that mismatch often remains invisible in aggregate return until late training stages. Early on, planner scaffolds can still improve exploration despite local infeasibility; later, the same infeasibility limits asymptotic performance by constraining policy updates to unrealistic trajectories. Effective diagnosis therefore needs intermediate metrics such as subgoal realizability, repair frequency, and deviation cost between proposed and executed plans. Without these diagnostics, systems may appear stable while accumulating structural debt.

6.2 Verifier exploitation and reward hacking

Verifier channels can be gamed when they encode shortcuts correlated with but not equivalent to task success. Agents then optimize verifier scores instead of environment objectives. This risk grows with verifier authority. Mitigations include disagreement-aware verifier ensembles, periodic grounding against raw environment return, and adversarial stress tests that explicitly target verifier vulnerabilities.

A practical mitigation is authority tapering: verifier influence is increased only in regions where calibration remains stable and reduced when disagreement spikes. Another is dual-trace auditing, where trajectories are scored both by verifiers and by delayed grounded outcomes to estimate proxy divergence. These controls do not eliminate exploitation, but they make it measurable and containable. They also support safer deployment by enabling automatic authority rollback when proxy drift is detected.

6.3 World-model hallucination and uncertainty propagation

World-model channels fail when imagined dynamics are overconfident under distribution shift. Hallucinated transitions can bias value targets and destabilize policy updates, especially when synthetic replay dominates training. Mitigations include uncertainty-weighted replay, capped synthetic budgets, and divergence monitoring between imagined and observed transitions.

An important nuance is error compounding depth. Even small one-step model errors can produce large long-horizon bias when rollouts are chained and then used for target construction. Depth-aware controls are therefore essential: shallow rollouts can be trusted more than deep rollouts unless confidence remains high across steps. Combining horizon-dependent confidence discounting with periodic grounded re-anchoring can materially reduce this failure mode without discarding the efficiency benefits of imagination.

6.4 Tool-mediated error cascades

Tool and memory channels can accumulate stale or incorrect context that persists across many decisions. Because downstream modules condition on the same context, a single early error can cascade through planning, verification, and control. Mitigations include schema validation, bounded memory windows, confidence-aware writes, and rollback policies triggered by inconsistency detection.

This cascade risk is amplified when tool outputs have heterogeneous reliability but are stored in a uniform memory format. A robust design treats provenance as first-class metadata, enabling policies to condition on source reliability and recency. In practice, provenance-aware memory reduces hidden-state leakage by preventing low-trust tool artifacts from receiving the same authority as grounded observations.

6.5 Compute-latency versus performance

Additional modules and deliberation loops can improve quality but increase inference and coordination cost. In time-constrained settings, over-deliberation can reduce effective utility despite higher nominal success. Mitigations include adaptive compute budgets, confidence-based early stopping, and module activation policies that reserve expensive channels for high-uncertainty states.

This trade-off should be framed as allocation, not minimization. The objective is not to use as little compute as possible, but to spend compute where it most improves trajectory quality. Policies that adaptively activate planners, verifiers, or world channels based on uncertainty can dominate fixed budgets by preserving responsiveness on easy states and investing in hard states. Reporting only average latency obscures this distinction; percentile latency and per-channel activation rates are more informative for system design.

System trade-offs are therefore unavoidable. Increasing module expressivity improves adaptability but can reduce stability if arbitration is under-specified. Tight modular interfaces improve interpretability but may constrain end-to-end adaptation. Conservative grounding improves robustness but can slow progress on sparse-reward tasks. The practical objective is not maximal module influence, but calibrated influence that improves learning while preserving trustworthy control.

A useful way to frame these trade-offs is through resource allocation under uncertainty. Each operator consumes compute and introduces estimation error. Increasing planner depth may improve branch quality but reduce interaction throughput. Increasing verifier strictness may suppress noise but remove useful exploratory traces. Increasing synthetic replay may improve coverage but inflate model bias. The best

Failure	Root cause	Symptom	Mitigation	Rep. refs
Planner-policy mismatch	Infeasible subgoals, weak controller grounding	Looping recovery, unstable option credit	Feasibility critics, online plan repair, adaptive authority	[1, 6]
Verifier exploitation	Proxy optimization of verifier signal	High verifier score, weak real return	Calibration, disagreement checks, grounding audits	[19, 20]
Reward misspecification	Misaligned shaped objectives	Reward hacking, poor transfer	Bounded shaping, objective consistency checks	[4, 37]
World-model hallucination	Overconfident synthetic dynamics	Value drift, deployment failure	Uncertainty weighting, synthetic budget limits	[16, 17]
Tool-memory cascade	Persistent noisy external context	Compounding multi-turn errors	Typed interfaces, rollback, memory freshness controls	[5, 12]
Compute overload	Excessive deliberation and channel orchestration	Latency spikes, unstable throughput	Adaptive budgets and selective module activation	[30, 31]

Table 4 T4. System-level failure modes, symptoms, and mitigation patterns.

operating point is therefore context-dependent and should be tuned with explicit diagnostics rather than fixed defaults.

Another trade-off is between local explainability and global adaptation. Highly modular systems are easier to inspect because each channel has explicit semantics. However, strict modular boundaries can prevent joint adaptation, especially when module errors are correlated. End-to-end methods can absorb correlations but are harder to debug and calibrate. Hybrid approaches that preserve modular observability while allowing limited cross-channel adaptation appear promising for practical deployment.

A final trade-off concerns safety margin versus asymptotic performance. Conservative arbitration and bounded shaping usually reduce worst-case failures, but may cap peak benchmark performance. In safety-critical settings, this trade-off is often desirable. Future work should formalize risk-sensitive objectives that optimize expected return subject to pathway-level reliability constraints.

These trade-offs imply that system design should include explicit operating regimes rather than single global settings. A deployment profile can specify how module authority, compute budget, and safety constraints are adjusted across exploration, refinement, and high-risk states. Regime-aware design converts fragile static tuning into policy-level adaptation and improves robustness when environments or tool ecosystems drift.

7 Research Agenda: Ten Open Problems

7.1 Open Problem 1: Calibrated verifier uncertainty

Verifier channels are now widely used for trajectory filtering and update weighting, yet most systems treat verifier scores as point estimates. This is insufficient in distribution-shifted regimes where score confidence and correctness decouple. What is missing is a standardized uncertainty model that integrates calibration diagnostics into the learning loop. A concrete direction is to couple verifiers with uncertainty heads and disagreement ensembles, then adapt weighting schedules based on uncertainty thresholds rather than raw scores.

An additional challenge is calibration transfer. A verifier calibrated on one domain can become poorly calibrated after prompt changes, tool updates, or environment shifts. Research should target lightweight online recalibration schemes that preserve stability without requiring full retraining. Confidence-aware rollback rules could provide immediate safeguards when calibration drifts beyond acceptable bounds.

7.2 Open Problem 2: Authority arbitration policies

Module authority is often fixed by design, even though module reliability varies over training and across states. Static authority can either underuse valuable guidance or overtrust brittle channels. The field needs arbitration policies that learn when modules should advise, influence, or abstain. Promising directions include meta-controllers trained on disagreement and regret signals, and safety-aware arbitration with explicit fallback to grounded environment reward channels.

An open question is how to evaluate arbitration quality independently of downstream policy quality. Direct return metrics confound arbitration with module strength. A better approach may combine intervention-level counterfactual tests with regret decomposition, measuring whether arbitration decisions improved outcomes relative to plausible alternatives.

7.3 Open Problem 3: Credit assignment under mixed supervision

Current theory does not adequately characterize how correlated auxiliary channels alter bias and variance in policy updates. Planner outputs, verifier scores, and shaped rewards are often treated independently despite shared error modes. A useful direction is formal analysis of coupled channels, including covariance-aware update estimators and bounds on misalignment risk under channel dependence.

This problem also has a practical side: existing implementations usually tune channel weights by heuristic sweeps. Theory that maps observable diagnostics to principled weight updates could reduce tuning cost and improve transfer. Such theory would help convert mixed-supervision RL from craft practice into a more predictable optimization discipline.

7.4 Open Problem 4: Uncertainty-aware world construction

Language-driven simulation and synthetic replay can improve efficiency, but uncertainty modeling remains underdeveloped. Many pipelines rely on heuristic filtering without principled uncertainty propagation into value targets. A concrete path forward is to combine epistemic uncertainty estimation with replay allocation policies that adapt synthetic influence to confidence and observed divergence from real rollouts.

Long-horizon tasks amplify this challenge because small model biases compound across simulated trajectories. Future work should study horizon-aware uncertainty controls that reduce synthetic influence as rollout depth increases, unless corroborated by grounded evidence. This could prevent optimistic rollouts from dominating updates in delayed-reward settings.

7.5 Open Problem 5: Robust planner-controller contracts

Planner-controller mismatches remain a major source of instability, especially in long-horizon tasks. Existing interfaces often specify what plans look like but not what feasibility guarantees are required. Research should target contract-aware hierarchical RL where planners provide confidence and constraint metadata, and controllers return structured feasibility feedback that updates planner behavior online.

Contract design may also benefit from domain-specific interface languages that encode hard constraints explicitly. In safety-critical domains, formal contract checking could prevent invalid plans from entering control loops, reducing reliance on downstream repair and improving predictable behavior.

7.6 Open Problem 6: Tool and memory reliability standards

Tool and memory modules effectively become part of state representation, yet reliability standards are inconsistent. Missing capabilities include principled memory lifecycle management, schema-level validation guarantees, and replay-safe handling of tool failures. A concrete direction is to formalize tool-memory state as a monitored subsystem with explicit error budgets and recovery protocols.

Open deployment settings further require provenance-aware memory updates. Systems should track whether memory content is observation-grounded, tool-derived, or model-generated, and condition trust accordingly. This distinction is often absent in current pipelines and contributes to hidden-state contamination.

7.7 Open Problem 7: Synthetic data governance for offline RL

Offline augmentation pipelines often improve benchmark scores, but provenance and contamination controls are still weak. Without governance, synthetic traces can leak benchmark artifacts or introduce confounding shortcuts. Future work should establish rigorous provenance tags, synthetic-to-real influence accounting, and benchmark protocols that isolate gains from synthetic generation versus improved policy learning.

Another unresolved issue is legal and ethical traceability for generated data in deployed systems. Governance frameworks should include reproducible generation logs

and auditable filtering criteria so that failures can be diagnosed and corrected without ambiguous data lineage.

7.8 Open Problem 8: Evaluation standardization across domains

Cross-paper comparison remains difficult because intervention contracts, tool permissions, and prompt policies are underreported. The field lacks a common reporting schema that captures channel-level behavior. A concrete direction is benchmark cards for LLM-in-RL systems that require pathway diagnostics, perturbation tests, and protocol disclosures in addition to final metrics.

Standardization should also define minimum stress-test suites and acceptable reporting granularity for long-horizon evaluation. Without agreed baselines, reported progress can reflect benchmark idiosyncrasies rather than genuine algorithmic advances.

7.9 Open Problem 9: Safety under open-ended deployment

As moduleized RL systems move toward real-world deployment, safety failures can arise from subtle channel interactions rather than obvious policy errors. Existing safeguards focus on isolated modules and miss cross-channel escalation. Research should prioritize runtime anomaly detection across supervision pathways, fail-safe authority reduction, and post-incident causal auditing of module interactions.

An important gap is proactive risk forecasting. Current systems largely detect failures after performance drops. Forecasting models that anticipate high-risk states from disagreement and uncertainty signatures could trigger preventive authority reductions before severe failures occur.

7.10 Open Problem 10: Scaling laws for modular RL systems

Scaling analysis often focuses on model parameters while ignoring interaction topology and authority allocation. In LLM-in-the-loop RL, performance may scale primarily with orchestration quality and compute distribution across channels. A key direction is module-aware scaling laws that jointly model planner depth, verifier reliability, simulation budget, and policy capacity under fixed compute constraints.

Beyond predictive curves, scaling work should provide actionable recipes for compute allocation under budget constraints. Practitioners need guidance on whether additional compute is better spent on policy capacity, verifier quality, planner depth, or simulation fidelity for a given task family.

These open problems share a common theme: reliability hinges on pathway-aware design. Future progress will likely come from methods that treat language modules as probabilistic supervisors with explicit uncertainty, bounded authority, and measurable causal impact on optimization.

Collectively, these directions point toward a transition from heuristic module composition to principled modular optimization. The most impactful advances may come from interfaces and diagnostics that make pathway behavior predictable across

domains, not from isolated gains in any single module. This shift would align methodological rigor with practical deployment needs and make long-horizon improvements more durable.

8 Conclusion

This survey presented a mechanism-first synthesis of how language modules improve reinforcement learning when embedded inside the RL loop. The central argument is that these systems are best understood as trajectory-level optimization architectures with heterogeneous supervision channels. Planner modules reshape trajectory priors, reward and verifier modules densify and filter supervision, world modules alter replay and planning distributions, and tool-memory modules reconstruct effective state under partial observability. Across domains, these interventions can produce substantial gains in long-horizon control and sample efficiency when channel authority is calibrated and grounded.

The same interventions also create systematic risks. Every auxiliary channel that reduces variance can introduce bias; every planner that improves decomposition can create feasibility mismatch; every synthetic replay mechanism that expands support can inject model error. Reliable design therefore requires explicit arbitration, pathway diagnostics, and stress-tested calibration rather than implicit trust in module outputs. This perspective shifts the field away from one-off module success stories toward integrated systems engineering for trustworthy credit assignment.

A mature research program in LLM-in-the-loop RL should prioritize three principles. First, authority should be adaptive and uncertainty-aware, with clear fallback to grounded reward channels. Second, evaluation should include protocol-level transparency and perturbation testing that exposes channel fragility. Third, optimization theory should model mixed-supervision pathways explicitly, enabling principled control of bias-variance trade-offs. With these foundations, language modules can evolve from heuristic assistants into reliable components for long-horizon, environment-grounded reinforcement learning.

In practical terms, the next phase of the field will likely be decided by engineering discipline as much as model capability. Systems that expose intervention contracts, quantify uncertainty, and maintain auditable supervision pathways will scale more reliably than systems that maximize short-term benchmark gains through opaque orchestration. The central opportunity is therefore not only to make RL agents more capable, but to make their improvement pathways measurable, controllable, and dependable over long horizons.

This perspective also reframes progress criteria for the community. Strong results should combine capability gains with pathway transparency: what changed in trajectory distribution, how update targets were altered, where uncertainty was introduced, and how authority was constrained when channels disagreed. Progress measured this way is slower but more durable, and it is better aligned with deployment needs in domains where long-horizon failures are costly.

Under this framing, the long-term value of LLM modules in RL is not merely that they add semantic fluency to agents, but that they provide controllable intervention interfaces for optimization. When these interfaces are calibrated, audited, and adaptively arbitrated, they can reduce horizon difficulty and improve robustness simultaneously. When they are opaque or over-authorized, they can create brittle systems that optimize proxies rather than task objectives. The central challenge for the field is therefore governance of supervision pathways: designing architectures where additional intelligence channels improve learning without undermining trustworthiness.

Appendix A Category-to-Paper Mapping

Table A1 groups all surveyed papers by mechanism category. Each paper is referenced by bibkey and short title.

Table A1: A1. Grouped category-to-paper mapping.

Category	Bibkey	Short title
Trajectory Prior Shaping	[45]	Online Decision Transformer
	[1]	Do As I Can, Not As I Say: Grounding Language in Robotic Affordances
	[48]	Demystifying Long Chain-of-Thought Reasoning in LLMs
	[49]	Conversational Planning for Personal Plans
	[50]	R1-Code-Interpreter: LLMs Reason with Code via Supervised and Multi-stage Reinforcement Learning
	[51]	MindFlow+: A Self-Evolving Agent for E-Commerce Customer Service
	[52]	Agentic Reinforced Policy Optimization
	[23]	Agent Lightning: Train ANY AI Agents with Reinforcement Learning
	[44]	ProMed: Shapley Information Gain Guided Reinforcement Learning for Proactive Medical LLMs
	[53]	The Landscape of Agentic Reinforcement Learning for LLMs: A Survey
	[54]	Reinforced Strategy Optimization for Conversational Recommender Systems via Network-of-Experts
	[55]	Scaling LLM Multi-turn RL with End-to-end Summarization-based Context Management
	[56]	Beyond Static LLM Policies: Imitation-Enhanced Reinforcement Learning for Recommendation
	[35]	Information Gain-based Policy Optimization: A Simple and Effective Approach for Multi-Turn LLM Agents
	[57]	PIGEON: VLM-Driven Object Navigation via Points of Interest Selection
	[11]	Agent-R1: Training Powerful LLM Agents with End-to-End Reinforcement Learning

Category	Bibkey	Short title
	[58]	LA-RL: Language Action-guided Reinforcement Learning with Safety Guarantees for Autonomous Highway Driving
	[9]	CoDA: A Context-Decoupled Hierarchical Agent with Reinforcement Learning
	[59]	Meta-RL Induces Exploration in Language Agents
	[26]	Graph-Symbolic Policy Enforcement and Control (G-SPEC): A Neuro-Symbolic Framework for Safe Agentic AI in 5G Autonomous Networks
	[60]	Towards AGI A Pragmatic Approach Towards Self Evolving Agent
	[10]	CREATE: Cross-Layer Resilience Characterization and Optimization for Efficient yet Reliable Embodied AI Systems
	[61]	Timely Machine: Awareness of Time Makes Test-Time Scaling Agentic
	[62]	EmboCoach-Bench: Benchmarking AI Agents on Developing Embodied Robots
	[24]	David vs. Goliath: Verifiable Agent-to-Agent Jailbreaking via Reinforcement Learning
	[63]	TrailBlazer: History-Guided Reinforcement Learning for Black-Box LLM Jailbreaking
	[64]	SeeUPO: Sequence-Level Agentic-RL with Convergence Guarantees
	[36]	AceGRPO: Adaptive Curriculum Enhanced Group Relative Policy Optimization for Autonomous Machine Learning Engineering
	[65]	CAPER: Constrained and Procedural Reasoning for Robotic Scientific Experiments
	[66]	LLM-Based Scientific Equation Discovery via Physics-Informed Token-Regularized Policy Optimization
	[28]	LACONIC: Length-Aware Constrained Reinforcement Learning for LLM
	[67]	Chasing Progress, Not Perfection: Revisiting Strategies for End-to-End LLM Plan Generation
	[68]	Deep learning-based robotic cloth manipulation applications: systematic review, challenges and opportunities for physical AI
	[69]	Robot Planning via LLM Proposals and Symbolic Verification
	[70]	Exploring the synergies between transfer in reinforcement learning and procedural content generation
Supervision Restructuring	[71]	Incentivizing Consistent, Effective and Scalable Reasoning Capability in Audio LLMs via Re...
	[20]	VI-CuRL: Stabilizing Verifier-Independent RL Reasoning via Confidence-Guided Variance Reduction
State & World Construction	[72]	History Compression via Language Models in Reinforcement Learning
	[15]	Do Embodied Agents Dream of Pixelated Sheep: Embodied Decision Making using Language Guided World Modelling

Category	Bibkey	Short title
	[73]	Guiding Pretraining in Reinforcement Learning with Large Language Models
	[74]	Reward Design with Language Models
	[7]	Skill Reinforcement Learning and Planning for Open-World Long-Horizon Tasks
	[75]	RL4F: Generating Natural Language Feedback with Reinforcement Learning for Repairing Model Outputs
	[3]	Voyager: An Open-Ended Embodied Agent with Large Language Models
	[38]	Self-Refined Large Language Model as Automated Reward Function Designer for Deep Reinforcement Learning in Robotics
	[4]	Eureka: Human-Level Reward Design via Coding Large Language Models
	[76]	Large Language Models as Generalizable Policies for Embodied Tasks
	[77]	Plug-and-Play Policy Planner for Large Language Model Powered Dialogue Agents
	[78]	Large Language Model as a Policy Teacher for Training Reinforcement Learning Agents
	[79]	Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects
	[80]	ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL
	[8]	Plan-Seq-Learn: Language Model Guided RL for Solving Long Horizon Robotics Tasks
	[42]	Asynchronous Large Language Model Enhanced Planner for Autonomous Driving
	[81]	SPELL: Self-Play Reinforcement Learning for Evolving Long-Context Language Models
	[82]	Don't Just Fine-tune the Agent, Tune the Environment
	[83]	Beyond Pipelines: A Survey of the Paradigm Shift toward Model-Native Agentic AI
	[84]	From Passive Metric to Active Signal: The Evolving Role of Uncertainty Quantification in Large Language Models
	[85]	ShopSimulator: Evaluating and Exploring RL-Driven LLM Agent for Shopping Assistants
	[22]	Mock Worlds, Real Skills: Building Small Agentic Language Models with Synthetic Tasks, Simulated Environments, and Rubric-Based Re...
	[18]	World Models as an Intermediary between Agents and the Real World
	[16]	World-Gymnast: Training Robots with Reinforcement Learning in a World Model
	[86]	STAR: Similarity-guided Teacher-Assisted Refinement for Super-Tiny Function Calling Models
	[17]	RISE: Self-Improving Robot Policy with Compositional World Model
	[87]	GigaBrain-0.5M*: a VLA That Learns From World Model-Based Reinforcement Learning

Category	Bibkey	Short title
	[40]	In-Context Autonomous Network Incident Response: An End-to-End Large Language Model Agent Approach
	[6]	HiPER: Hierarchical Reinforcement Learning with Explicit Credit Assignment for Large Language Model Agents
	[2]	Code as Policies: Language Model Programs for Embodied Control
	[88]	On the Prospects of Incorporating Large Language Models (LLMs) in Automated Planning and Scheduling (APS)
	[39]	A Survey of Robot Intelligence with Large Language Models
	[43]	SPARE: Single-Pass Annotation with Reference-Guided Evaluation for Automatic Process Supervision and Reward Modelling
Search & Deliberation Hybrids	[21]	Code Evolution for Control: Synthesizing Policies via LLM-Driven Evolutionary Search
	[89]	Training Multi-Turn Search Agent via Contrastive Dynamic Branch Sampling
	[19]	Adaptive Uncertainty-Aware Tree Search for Robust Reasoning
	[90]	SIGHT: Reinforcement Learning with Self-Evidence and Information-Gain Diverse Branching for Search Agent
	[91]	REDSearcher: A Scalable and Cost-Efficient Framework for Long-Horizon Search Agents
	[31]	Navigating the Alpha Jungle: An LLM-Powered MCTS Framework for Formulaic Alpha Factor Mining
Data & Offline Augmentation	[32]	Unleashing the Power of Pre-trained Language Models for Offline Reinforcement Learning
	[92]	Planning without Search: Refining Frontier LLMs with Offline Goal-Conditioned RL
	[93]	RoboMIND 2.0: A Multimodal, Bimanual Mobile Manipulation Dataset for Generalizable Embodied Intelligence
	[94]	UltraLogic: Enhancing LLM Reasoning through Large-Scale Data Synthesis and Bipolar Float Reward
	[27]	STO-RL: Offline RL under Sparse Rewards via LLM-Guided Subgoal Temporal Order
	[95]	Attention-Based Offline Reinforcement Learning and Clustering for Interpretable Sepsis Treatment
	[96]	Reinforcement Learning via Self-Distillation
	[97]	DCoPilot: Generative AI-Empowered Policy Adaptation for Dynamic Data Center Operations
	[98]	TMS: Trajectory-Mixed Supervision for Reward-Free, On-Policy SFT
	[99]	medR: Reward Engineering for Clinical Offline Reinforcement Learning via Tri-Drive Potential Functions
	[100]	Intelligent Agents for Data Exploration

Category	Bibkey	Short title
Tool & Memory Mediation	[101]	Multimodal Web Navigation with Instruction-Finetuned Foundation Models
	[37]	Text2Reward: Reward Shaping with Language Models for Reinforcement Learning
	[5]	AutoWebGLM: A Large Language Model-based Web Navigating Agent
	[102]	WebRL: Training LLM Web Agents via Self-Evolving Online Curriculum Reinforcement Learning
	[103]	WorkForceAgent-R1: Incentivizing Reasoning Capability in LLM-based Web Agents via Reinforcement Learning
	[104]	WebExplorer: Explore and Evolve for Training Long-Horizon Web Agents
	[105]	Tool-R1: Sample-Efficient Reinforcement Learning for Agentic Tool Use
	[106]	Process-Supervised Reinforcement Learning for Interactive Multimodal Tool-Use Agents
	[107]	MENTOR: A Reinforcement Learning Framework for Enabling Tool Use in Small Models via Teacher-Optimized Rewards
	[25]	AgentFold: Long-Horizon Web Agents with Proactive Context Management
	[108]	GAP: Graph-Based Agent Planning with Parallel Tool Use and Reinforcement Learning
	[14]	WebGym: Scaling Training Environments for Visual Web Agents with Realistic Tasks
	[109]	SCRIBE: Structured Mid-Level Supervision for Tool-Using Language Models
	[13]	The Confidence Dichotomy: Analyzing and Mitigating Miscalibration in Tool-Use Agents
	[110]	Teaching LLMs to Learn Tool Trialing and Execution through Environment Interaction
	[29]	DynaWeb: Model-Based Reinforcement Learning of Web Agents
	[111]	How Far Are LLMs from Professional Poker Players? Revisiting Game-Theoretic Reasoning with Agentic Tool Use
	[12]	Reasoning and Tool-use Compete in Agentic RL:From Quantifying Interference to Disentangled Tuning
	[112]	OpAgent: Operator Agent for Web Navigation
	[113]	ScribeAgent: Towards Specialized Web Agents Using Production-Scale Workflow Data
Multi-agent / Interactive	[33]	Language Instructed Reinforcement Learning for Human-AI Coordination
	[114]	SMART-LLM: Smart Multi-Agent Robot Task Planning using Large Language Models
	[46]	LLM-Coordination: Evaluating and Analyzing Multi-agent Coordination Abilities in Large Language Models

Category	Bibkey	Short title
	[115]	OWL: Optimized Workforce Learning for General Multi-Agent Assistance in Real-World Task Automation
	[116]	Chain-of-Agents: End-to-End Agent Foundation Models via Multi-Agent Distillation and Agentic RL
	[117]	Mixture-of-Minds: Multi-Agent Reinforcement Learning for Table Understanding
	[41]	Learning-based Cooperative Robotic Paper Wrapping: A Unified Control Policy with Residual Force Control
	[118]	MarsRL: Advancing Multi-Agent Reasoning System via Reinforcement Learning with Agentic Pipeline Parallelism
	[119]	Bilevel Optimization for Covert Memory Tampering in Heterogeneous Multi-Agent Architectures (XAMT)
	[120]	ResMAS: Resilience Optimization in LLM-based Multi-agent Systems
	[121]	FinEvo: From Isolated Backtests to Ecological Market Games for Multi-Agent Financial Strategy Evolution
	[122]	WideSeek-R1: Exploring Width Scaling for Broad Information Seeking via Multi-Agent Reinforcement Learning
	[123]	CoLF: Learning Consistent Leader-Follower Policies for Vision-Language-Guided Multi-Robot Cooperative Transport
	[30]	MARTI-MARS 2 : Scaling Multi-Agent Self-Search via Reinforcement Learning for Code Generation
	[34]	Who Deserves the Reward? SHARP: Shapley Credit-based Optimization for Multi-Agent System
	[47]	The Five Ws of Multi-Agent Communication: Who Talks to Whom, When, What, and Why – A Survey from MARL to Emergent Language and LL...
	[124]	Theory of Mind for Multi-Agent Collaboration via Large Language Models
	[125]	Adaptive Regulation via Dual-Layer Evolution (ARDE): A Multi-Agent Approach to Balancing Efficiency, Fairness, and Diversity in Cr...

References

[1] Ahn, M.J., Brohan, A., Brown, N., Chebotar, Y., Cortes, O.A.C., David, B., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Ho, D.W.C., Hsu, J., Ibarz, J., Ichter, B., Irpan, A., Jang, E.B., Ruano, R.J., Jeffrey, K., Jesmonth, S., Joshi, N.J., Julian, R.R., Kalashnikov, D., Kuang, Y., Lee, K.-H., Levine, S., Lu, Y., Luu, L., Parada, C., Pastor, P., Quiambao, J., Rao, K., Rettinghouse, J., Reyes, D., Sermanet, P., Sievers, N., Tan, C., Toshev, A., Vanhoucke, V., Xia, F., Xiao, T., Xu, P., Xu, S., Yan, M., Yan, Mengyuan, Zeng, Andy: Do As I

Can, Not As I Say: Grounding Language in Robotic Affordances. arXiv (Cornell University) (2022). https://arxiv.org/abs/2204.01691

[2] Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., Zeng, A.: Code as Policies: Language Model Programs for Embodied Control. Unknown venue (2023). https://doi.org/10.1109/icra48891.2023.10160591 . https://doi.org/10.1109/icra48891.2023.10160591

[3] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., Anandkumar, A.: Voyager: An Open-Ended Embodied Agent with Large Language Models. ArXiv (2023). https://arxiv.org/abs/2305.16291

[4] Ma, Y.J., Liang, W., Wang, G., Huang, D.-A., Bastani, O., Jayaraman, D., Zhu, Y., Fan, L., Anandkumar, A.: Eureka: Human-Level Reward Design via Coding Large Language Models. ArXiv (2023). https://arxiv.org/abs/2310.12931

[5] Lai, H., Liu, X., Iong, I.L., Yao, S., Chen, Y., Shen, P., Yu, H., Zhang, H., Zhang, X., Dong, Y., Tang, J.: AutoWebGLM: A Large Language Model-based Web Navigating Agent. ArXiv (2024). https://arxiv.org/abs/2404.03648

[6] Peng, J., Liu, Y., Zhou, R., Fleming, C., Wang, Z., Garcia, A., Hong, M.: HiPER: Hierarchical Reinforcement Learning with Explicit Credit Assignment for Large Language Model Agents. ArXiv (2026). https://arxiv.org/abs/2602.16165

[7] Yuan, H., Zhang, C., Wang, H., Xie, F., Cai, P., Dong, H., Lu, Z.: Skill Reinforcement Learning and Planning for Open-World Long-Horizon Tasks. ArXiv (2023). https://arxiv.org/abs/2303.16563

[8] Dalal, M., Chiruvolu, T., Chaplot, D., Salakhutdinov, R.: Plan-Seq-Learn: Language Model Guided RL for Solving Long Horizon Robotics Tasks. ArXiv (2024). https://arxiv.org/abs/2405.01534

[9] Liu, X., Feng, J., Zhuang, Z., Zhao, J., Que, M., Li, J., Wang, D., Tong, H., Chen, Y., Li, P.: CoDA: A Context-Decoupled Hierarchical Agent with Reinforcement Learning. ArXiv (2025). https://arxiv.org/abs/2512.12716

[10] Xie, T., Qi, Y., Wen, J., Wan, Z., Dong, Y., Wang, Z., Cai, S., Liang, Y., Jia, T., wang, Wang, R., Li, M.: CREATE: Cross-Layer Resilience Characterization and Optimization for Efficient yet Reliable Embodied AI Systems. arXiv (Cornell University) (2026). https://arxiv.org/abs/2601.14140

[11] Cheng, M., Ouyang, J., Yu, S., Yan, R., Luo, Y., Liu, Z., Wang, D., Liu, Q., Chen, E.: Agent-R1: Training Powerful LLM Agents with End-to-End Reinforcement Learning. ArXiv (2025). https://arxiv.org/abs/2511.14460

[12] Li, Y., Yi, M., Li, X., Fan, J., Jiang, F., Chen, B., Li, P., Song, J., Zhang, T.: Reasoning and Tool-use Compete in Agentic RL:From Quantifying Interference

to Disentangled Tuning. ArXiv (2026). https://arxiv.org/abs/2602.00994

[13] Xuan, W., Zeng, Q., Qi, H., Xiao, Y., Wang, J., Yokoya, N.: The Confidence Dichotomy: Analyzing and Mitigating Miscalibration in Tool-Use Agents. ArXiv (2026). https://arxiv.org/abs/2601.07264

[14] Bai, H., Taymanov, A., Zhang, T., Kumar, A., Whitehead, S.: WebGym: Scaling Training Environments for Visual Web Agents with Realistic Tasks. ArXiv (2026). https://arxiv.org/abs/2601.02439

[15] Nottingham, K., Ammanabrolu, P., Suhr, A., Choi, Y., Hajishirzi, H., Singh, S., Fox, R.: Do Embodied Agents Dream of Pixelated Sheep: Embodied Decision Making using Language Guided World Modelling. ArXiv (2023). https://arxiv.org/abs/2301.12050

[16] Sharma, A.K., Sun, Y., Lu, N., Zhang, Y., Liu, J., Yang, S.: World-Gymnast: Training Robots with Reinforcement Learning in a World Model. ArXiv (2026). https://arxiv.org/abs/2602.02454

[17] Yang, J., Lin, K., Li, J., Zhang, W., Lin, T., Wu, L., Su, Z., Zhao, H., Zhang, Y.-Q., Chen, L., Luo, P., Yue, X., Li, H.: RISE: Self-Improving Robot Policy with Compositional World Model. ArXiv (2026). https://arxiv.org/abs/2602.11075

[18] Yang, S.: World Models as an Intermediary between Agents and the Real World. ArXiv (2026). https://arxiv.org/abs/2602.00785

[19] Song, Z., Ma, Z., Qiang, W., Zheng, C., Hua, G.: Adaptive Uncertainty-Aware Tree Search for Robust Reasoning. ArXiv (2026). https://arxiv.org/abs/2602.06493

[20] Cai, X.-Q., Sugiyama, M.: VI-CuRL: Stabilizing Verifier-Independent RL Reasoning via Confidence-Guided Variance Reduction. ArXiv (2026). https://arxiv.org/abs/2602.12579

[21] Guo, P., Li, C., Feng, Y., Zhang, C.: Code Evolution for Control: Synthesizing Policies via LLM-Driven Evolutionary Search. ArXiv.org (2026). https://arxiv.org/abs/2601.06845

[22] Lü, Y.-J., Wang, C., Shen, L., Huang, J., Xu, T.: Mock Worlds, Real Skills: Building Small Agentic Language Models with Synthetic Tasks, Simulated Environments, and Rubric-Based Rewards. ArXiv.org (2026). https://arxiv.org/abs/2601.22511

[23] Luo, X., Zhang, Y., He, Z., Wang, Z., Zhao, S., Li, D., Qiu, L.K., Yang, Y.: Agent Lightning: Train ANY AI Agents with Reinforcement Learning. ArXiv (2025). https://arxiv.org/abs/2508.03680

[24] Nellessen, S., Kachman, T.: David vs. Goliath: Verifiable Agent-to-Agent Jailbreaking via Reinforcement Learning. ArXiv (2026). https://arxiv.org/abs/2602.02395

[25] Ye, R., Zhang, Z., Li, K., Yin, H., Tao, Z., Zhao, Y., Su, L., Zhang, L., Qiao, Z., Wang, X., Xie, P., Huang, F., Chen, S., Zhou, J., Jiang, Y.: AgentFold: Long-Horizon Web Agents with Proactive Context Management. ArXiv (2025). https://arxiv.org/abs/2510.24699

[26] Vijay, D., Ethiraj, V.: Graph-Symbolic Policy Enforcement and Control (G-SPEC): A Neuro-Symbolic Framework for Safe Agentic AI in 5G Autonomous Networks. ArXiv (2025). https://arxiv.org/abs/2512.20275

[27] Gu, C., Pan, Y., Xiong, H., Chen, Y.: STO-RL: Offline RL under Sparse Rewards via LLM-Guided Subgoal Temporal Order. ArXiv.org (2026). https://arxiv.org/abs/2601.08107

[28] Liu, C., Zhao, Y., Liu, L., Ye, Y., Szepesvári, C., Yang, L.F.: LACONIC: Length-Aware Constrained Reinforcement Learning for LLM. ArXiv (2026). https://arxiv.org/abs/2602.14468

[29] Ding, H., Liu, P., Wang, J., Ji, Z., Cao, M., Zhang, R., Ai, L., Yang, E., Shi, T., Yu, L.: DynaWeb: Model-Based Reinforcement Learning of Web Agents. ArXiv (2026). https://arxiv.org/abs/2601.22149

[30] Wang, S., Li, P., Fu, Y., Liu, K., Li, F., Liu, Y., Sun, X., Li, Z., Zhao, S., Zhao, J., Tian, K., Li, D., Gao, J., Zhang, Y., Chen, Y., Li, Y., Li, Z., Zhang, W., Ye, P., Hu, S., Bai, L., Zhou, B., Zhang, K., Qi, B.: MARTI-MARS^2: Scaling Multi-Agent Self-Search via Reinforcement Learning for Code Generation. ArXiv (2026). https://arxiv.org/abs/2602.07848

[31] Artificial Intelligence 2026, A., Duan, Y., Jian, L., Shi, Y.: Navigating the Alpha Jungle: An LLM-Powered MCTS Framework for Formulaic Alpha Factor Mining. Underline Science Inc. (2026). https://doi.org/10.48448/q4pk-2g71 . https://doi.org/10.48448/q4pk-2g71

[32] Shi, R., Liu, Y., Ze, Y., Du, S.S., Xu, H.: Unleashing the Power of Pre-trained Language Models for Offline Reinforcement Learning. ArXiv (2023). https://arxiv.org/abs/2310.20587

[33] Hu, H., Sadigh, D.: Language Instructed Reinforcement Learning for Human-AI Coordination. ArXiv (2023). https://arxiv.org/abs/2304.07297

[34] Li, Y., Zhang, X., Lu, W., Tang, Z., Wu, M., Luo, H., Wu, T., Peng, Z., Mi, H., Feng, Y., Tan, N., Huang, C., Chen, H., Shen, L.: Who Deserves the Reward? SHARP: Shapley Credit-based Optimization for Multi-Agent System. ArXiv (2026). https://arxiv.org/abs/2602.08335

[35] Wang, G., Dai, S., Ye, G., Gan, Z., Yao, W., Deng, Y., Wu, X., Ying, Z.: Information Gain-based Policy Optimization: A Simple and Effective Approach for Multi-Turn LLM Agents. ArXiv (2025). https://arxiv.org/abs/2510.14967

[36] Cai, Y., Liu, Z., Zhu, X., Wang, C., Chen, J., Wang, H., Wang, W.-C., Jin, D., Chen, S.: AceGRPO: Adaptive Curriculum Enhanced Group Relative Policy Optimization for Autonomous Machine Learning Engineering. ArXiv (2026). https://arxiv.org/abs/2602.07906

[37] Xie, T., Zhao, S., Wu, C., Liu, Y., Luo, Q., Zhong, V.W., Yang, Y., Yu, C.: Text2Reward: Reward Shaping with Language Models for Reinforcement Learning. arXiv (Cornell University) (2023). https://arxiv.org/abs/2309.11489

[38] Song, J., Zhou, Z., Liu, J., Fang, C., Shu, Z., Ma, L.: Self-Refined Large Language Model as Automated Reward Function Designer for Deep Reinforcement Learning in Robotics. arXiv (Cornell University) (2023). https://arxiv.org/abs/2309.06687

[39] Jeong, H., Lee, H., Kim, C., Shin, S.: A Survey of Robot Intelligence with Large Language Models. Applied Sciences (2024). https://doi.org/10.3390/app14198868 . https://doi.org/10.3390/app14198868

[40] Gao, Y., Hammar, K., Li, T.: In-Context Autonomous Network Incident Response: An End-to-End Large Language Model Agent Approach. ArXiv (2026). https://arxiv.org/abs/2602.13156

[41] Ali, R.S., Beltran-Hernandez, C.C., Wan, W., Harada, K.: Learning-based Cooperative Robotic Paper Wrapping: A Unified Control Policy with Residual Force Control. arXiv (Cornell University) (2025). https://arxiv.org/abs/2511.03181

[42] Chen, Y., Ding, Z.-h., Wang, Z., Wang, Y., Zhang, L., Liu, S.: Asynchronous Large Language Model Enhanced Planner for Autonomous Driving. ArXiv (2024). https://arxiv.org/abs/2406.14556

[43] Artificial Intelligence 2026, A., Gurevych, I., Rizvi, M.I.H., Zhu, X.: SPARE: Single-Pass Annotation with Reference-Guided Evaluation for Automatic Process Supervision and Reward Modelling. Underline Science Inc. (2026). https://doi.org/10.48448/0js4-fm43 . https://doi.org/10.48448/0js4-fm43

[44] Ding, H., Huang, B., Fang, Y., Liao, W., Jiang, X., Li, Z., Zhao, J., Wang, Y.: ProMed: Shapley Information Gain Guided Reinforcement Learning for Proactive Medical LLMs. ArXiv (2025). https://arxiv.org/abs/2508.13514

[45] Zheng, Q., Zhang, A., Grover, A.: Online Decision Transformer. arXiv (Cornell University) (2022). https://arxiv.org/abs/2202.05607

[46] Agashe, S., Fan, Y., Reyna, A., Wang, X.E.: LLM-Coordination: Evaluating and

Analyzing Multi-agent Coordination Abilities in Large Language Models. ArXiv (2023). https://arxiv.org/abs/2310.03903

[47] Chen, J., Yang, H., Liu, Z., Joe-Wong, C.: The Five Ws of Multi-Agent Communication: Who Talks to Whom, When, What, and Why – A Survey from MARL to Emergent Language and LLMs. ArXiv (2026). https://arxiv.org/abs/2602.11583

[48] Yeo, E., Tong, Y., Niu, M., Neubig, G., Yue, X.: Demystifying Long Chain-of-Thought Reasoning in LLMs. ArXiv (2025). https://arxiv.org/abs/2502.03373

[49] Christakopoulou, K., Qu, I., Canny, J., Goodridge, A., Adams, C., Chen, M., Matarić, M.: Conversational Planning for Personal Plans. ArXiv (2025). https://arxiv.org/abs/2502.19500

[50] Chen, Y., Liu, Y., Zhou, J., Hao, Y., Wang, J., Zhang, Y., Li, N., Fan, C.: R1-Code-Interpreter: LLMs Reason with Code via Supervised and Multi-stage Reinforcement Learning. ArXiv (2025). https://arxiv.org/abs/2505.21668

[51] Gong, M., Huang, X., Xu, Z., Asari, V.K.: MindFlow+: A Self-Evolving Agent for E-Commerce Customer Service. ArXiv (2025). https://arxiv.org/abs/2507.18884

[52] Dong, G., Mao, H., Ma, K., Bao, L., Chen, Y., Wang, Z., Chen, Z., Du, J., Wang, H., Zhang, F., Zhou, G., Zhu, Y., Wen, J.-R., Dou, Z.: Agentic Reinforced Policy Optimization. ArXiv (2025). https://arxiv.org/abs/2507.19849

[53] Zhang, G., Geng, H., Yu, X., Yin, Z., Zhang, Z., Tan, Z., Zhou, H., Li, Z., Xue, X., Li, Y., Zhou, Y., Chen, Y., Zhang, C., Fan, Y., Wang, Z., Huang, S., Piedrahita-Velez, F., Liao, Y., Wang, H., Yang, M., Ji, H., Wang, J., Yan, S., Torr, P., Bai, L.: The Landscape of Agentic Reinforcement Learning for LLMs: A Survey. ArXiv (2025). https://arxiv.org/abs/2509.02547

[54] Zhao, X., Yan, M., Zhang, Y., Deng, Y., Wang, J., Zhu, F., Qiu, Y., Cheng, H., Chua, T.-S.: Reinforced Strategy Optimization for Conversational Recommender Systems via Network-of-Experts. ArXiv (2025). https://arxiv.org/abs/2509.26093

[55] Lu, M., Sun, W., Du, W., Ling, Z., Yao, X., Liu, K., Chen, J.: Scaling LLM Multi-turn RL with End-to-end Summarization-based Context Management. ArXiv (2025). https://arxiv.org/abs/2510.06727

[56] Zhang, Y., Xie, L., Qiu, R., Liu, J., Wang, S.: Beyond Static LLM Policies: Imitation-Enhanced Reinforcement Learning for Recommendation. ArXiv (2025). https://arxiv.org/abs/2510.13229

[57] Peng, C., Zhang, Z., Chi, C., Wei, X., Zhang, Y., Wang, H., Wang, P., Wang,

Z., Liu, J., Zhang, S.: PIGEON: VLM-Driven Object Navigation via Points of Interest Selection. arXiv (Cornell University) (2025). https://arxiv.org/abs/2511.13207

[58] Shu, Y., Xu, J., Tang, J., Gao, R.L., Sun, C.: LA-RL: Language Action-guided Reinforcement Learning with Safety Guarantees for Autonomous Highway Driving. arXiv (Cornell University) (2025). https://arxiv.org/abs/2512.05686

[59] Jiang, Y., Jiang, L., Teney, D., Moor, M., Brbic, M.: Meta-RL Induces Exploration in Language Agents. ArXiv (2025). https://arxiv.org/abs/2512.16848

[60] Kar, I., Zonunpuia, S., Ralte, Z.: Towards AGI A Pragmatic Approach Towards Self Evolving Agent. ArXiv (2026). https://arxiv.org/abs/2601.11658

[61] Ma, Y., Li, L., chen, Y., Li, P., Li, X., Guo, Q., Lin, D., Chen, K.: Timely Machine: Awareness of Time Makes Test-Time Scaling Agentic. ArXiv (2026). https://arxiv.org/abs/2601.16486

[62] Lei, Z., Liu, G., Zhang, Y., Liu, Q., Wen, C., Zhang, S., Lian, W., Chen, S.: EmboCoach-Bench: Benchmarking AI Agents on Developing Embodied Robots. ArXiv (2026). https://arxiv.org/abs/2601.21570

[63] Yoon, S.-H., Qian, R., Zhao, M., Li, W., Wang, M.: TrailBlazer: History-Guided Reinforcement Learning for Black-Box LLM Jailbreaking. ArXiv (2026). https://arxiv.org/abs/2602.06440

[64] Hu, T., Fu, Q., Chen, Y., Liu, Z., Ding, B.: SeeUPO: Sequence-Level Agentic-RL with Convergence Guarantees. ArXiv (2026). https://arxiv.org/abs/2602.06554

[65] Yang, J., Hou, J., Yu, X., He, W., Wu, Y.: CAPER: Constrained and Procedural Reasoning for Robotic Scientific Experiments. ArXiv (2026). https://arxiv.org/abs/2602.09367

[66] Wang, B., Li, K., Liu, T., Li, C., Wang, J., Zhang, Y., Cheng, J.: LLM-Based Scientific Equation Discovery via Physics-Informed Token-Regularized Policy Optimization. ArXiv (2026). https://arxiv.org/abs/2602.10576

[67] Huang, S., Cohn, T., Lipovetzky, N.: Chasing Progress, Not Perfection: Revisiting Strategies for End-to-End LLM Plan Generation. Proceedings of the International Conference on Automated Planning and Scheduling (2025). https://doi.org/10.1609/icaps.v35i1.36119 . https://doi.org/10.1609/icaps.v35i1.36119

[68] Gu, N., Hayashibe, M., Kutsuzawa, K., Yu, H.: Deep learning-based robotic cloth manipulation applications: systematic review, challenges and opportunities for physical AI. Frontiers in Robotics and AI (2026). https://doi.org/10.3389/frobt.2026.1752914 . https://doi.org/10.3389/frobt.2026.1752914

[69] Pesjak, D., Žabkar, J.: Robot Planning via LLM Proposals and Symbolic Verification. Machine Learning and Knowledge Extraction (2026). https://doi.org/10.3390/make8010022 . https://doi.org/10.3390/make8010022

[70] Müller-Brockhausen, Matthias: Exploring the synergies between transfer in reinforcement learning and procedural content generation. Unknown venue (2025). https://doi.org/10.60602/1887/4282228 . https://doi.org/10.60602/1887/4282228

[71] Fan, J., Ren, R., Li, J., Pandey, R., Shivakumar, P.G., Bulyko, I., Gandhe, A., Liu, G., Gu, Y.: Incentivizing Consistent, Effective and Scalable Reasoning Capability in Audio LLMs via Reasoning Process Rewards. ArXiv (2025). https://arxiv.org/abs/2510.20867

[72] Paischer, F., Adler, T., Patil, V., Bitto-Nemling, A., Holzleitner, M., Lehner, S., Eghbal-zadeh, H., Hochreiter, S.: History Compression via Language Models in Reinforcement Learning. arXiv (Cornell University) (2022). https://arxiv.org/abs/2205.12258

[73] Du, Y., Watkins, O., Wang, Z., Colas, C., Darrell, T., Abbeel, P., Gupta, A., Andreas, J.: Guiding Pretraining in Reinforcement Learning with Large Language Models. ArXiv (2023). https://arxiv.org/abs/2302.06692

[74] Kwon, M., Xie, S.M., Bullard, K., Sadigh, D.: Reward Design with Language Models. arXiv (Cornell University) (2023). https://arxiv.org/abs/2303.00001

[75] Akyürek, A.F., Akyürek, E., Madaan, A., Kalyan, A., Clark, P., Wijaya, D., Tandon, N.: RL4F: Generating Natural Language Feedback with Reinforcement Learning for Repairing Model Outputs. ArXiv (2023). https://arxiv.org/abs/2305.08844

[76] Szot, A., Schwarzer, M., Agrawal, H., Mazoure, B., Talbott, W., Metcalf, K., Mackraz, N., Hjelm, D., Toshev, A.: Large Language Models as Generalizable Policies for Embodied Tasks. ArXiv (2023). https://arxiv.org/abs/2310.17722

[77] Deng, Y., Zhang, W., Lam, W., Ng, S.-K., Chua, T.-S.: Plug-and-Play Policy Planner for Large Language Model Powered Dialogue Agents. ArXiv (2023). https://arxiv.org/abs/2311.00262

[78] Zhou, Z., Hu, B., Zhao, C., Zhang, P., Liu, B.: Large Language Model as a Policy Teacher for Training Reinforcement Learning Agents. ArXiv (2023). https://arxiv.org/abs/2311.13373

[79] Cheng, Y., Zhang, C., Zhang, Z., Meng, X., Hong, S., Li, W., Wang, Z., Wang, Z., Yin, F., Zhao, J., He, X.: Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects. ArXiv (2024). https://arxiv.org/abs/2401.03428

[80] Zhou, Y., Zanette, A., Pan, J., Levine, S., Kumar, A.: ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL. ArXiv (2024). https://arxiv.org/abs/2402.19446

[81] Yang, Z., Shen, W., Li, C., Chen, R., Wan, F., Yan, M., Quan, X., Huang, F.: SPELL: Self-Play Reinforcement Learning for Evolving Long-Context Language Models. ArXiv (2025). https://arxiv.org/abs/2509.23863

[82] Lu, S., Wang, Z., Zhang, H., Wu, Q., Gan, L., Zhuang, C., Gu, J., Lin, T.: Don't Just Fine-tune the Agent, Tune the Environment. ArXiv (2025). https://arxiv.org/abs/2510.10197

[83] Sang, J., Xiao, J., Han, J., Chen, J., Chen, X., Wei, S., Sun, Y., Wang, Y.: Beyond Pipelines: A Survey of the Paradigm Shift toward Model-Native Agentic AI. ArXiv (2025). https://arxiv.org/abs/2510.16720

[84] Zhang, J., Cui, W., Li, Z., Huang, L., Malin, B., Xiong, C., Wu, C.-S.: From Passive Metric to Active Signal: The Evolving Role of Uncertainty Quantification in Large Language Models. ArXiv (2026). https://arxiv.org/abs/2601.15690

[85] Wang, P., Wu, Y., Song, X., Wang, W., Chen, G., Li, Z., Yan, K., Deng, K., Liu, Q., Zhao, S., Xiong, S., Liu, X., Chen, X., Deng, W., Su, W., Zheng, B.: ShopSimulator: Evaluating and Exploring RL-Driven LLM Agent for Shopping Assistants. ArXiv (2026). https://arxiv.org/abs/2601.18225

[86] Ni, J., Pu, J., Yang, Z., Luo, J., Hu, C.: STAR: Similarity-guided Teacher-Assisted Refinement for Super-Tiny Function Calling Models. ArXiv (2026). https://arxiv.org/abs/2602.03022

[87] Team, G., Wang, B., Ni, C., Huang, G., Zhao, G., Li, H., Li, J., Lv, J., Liu, J., Feng, L., Yu, M., Li, P., Deng, Q., Liu, T., Zhou, X., Chen, X., Wang, X., Wang, Y., Li, Y., Nie, Y., Li, Y., Zhou, Y., Ye, Y., Liu, Z., Zhu, Z.: GigaBrain-0.5M*: a VLA That Learns From World Model-Based Reinforcement Learning. ArXiv (2026). https://arxiv.org/abs/2602.12099

[88] Pallagani, V., Muppasani, B., Roy, K., Fabiano, F., Loreggia, A., Murugesan, K., Srivastava, B., Rossi, F., Horesh, L., Sheth, A.: On the Prospects of Incorporating Large Language Models (LLMs) in Automated Planning and Scheduling (APS). Proceedings of the International Conference on Automated Planning and Scheduling (2024). https://doi.org/10.1609/icaps.v34i1.31503 . https://doi.org/10.1609/icaps.v34i1.31503

[89] Zhao, Y., Huang, W., Wang, S., Zhao, R., Chen, C., Shu, Y., Qin, C.: Training Multi-Turn Search Agent via Contrastive Dynamic Branch Sampling. ArXiv (2026). https://arxiv.org/abs/2602.03719

[90] Zhong, W., Yang, J., Wu, Y., Liu, Y., Yao, J., Kuang, K.: SIGHT: Reinforcement

Learning with Self-Evidence and Information-Gain Diverse Branching for Search Agent. ArXiv (2026). https://arxiv.org/abs/2602.11551

[91] Chu, Z., Wang, X., Hong, J., Fan, H., Huang, Y., Yang, Y., Xu, G., Zhao, C., Xiang, C., Hu, S., Kuang, D., Liu, M., Qin, B., Yu, X.: REDSearcher: A Scalable and Cost-Efficient Framework for Long-Horizon Search Agents. ArXiv (2026). https://arxiv.org/abs/2602.14234

[92] Hong, J., Dragan, A., Levine, S.: Planning without Search: Refining Frontier LLMs with Offline Goal-Conditioned RL. ArXiv (2025). https://arxiv.org/abs/2505.18098

[93] Hou, C., Wu, K., Liu, J., Che, Z., Wu, D., Liao, F., Li, G., He, J., Feng, Q., Jin, Z., Gu, C., Liu, Z., Han, N., Mi, X., Lv, Y., Fu, Y., Dai, G., Gu, L., Li, T., Zhang, Y., Zhang, Y., Wang, X., Fan, S., Li, M., Zhao, Z., Liu, N., Xu, Z., Ren, P.W., Ji, J., Liu, H., Cheng, K., Zhang, S., Tang, J.: RoboMIND 2.0: A Multimodal, Bimanual Mobile Manipulation Dataset for Generalizable Embodied Intelligence. ArXiv.org (2025). https://arxiv.org/abs/2512.24653

[94] Liu, Y., Liu, Y., Li, Z., Huang, Y., Feng, X., Hu, Z., Hu, J., Yan, J., Lian, F., Liu, Y.: UltraLogic: Enhancing LLM Reasoning through Large-Scale Data Synthesis and Bipolar Float Reward. ArXiv (2026). https://arxiv.org/abs/2601.03205

[95] Kumar, P., Saran, V., Patel, D., Kulkarni, N., Vereshchaka, A.: Attention-Based Offline Reinforcement Learning and Clustering for Interpretable Sepsis Treatment. ArXiv.org (2026). https://arxiv.org/abs/2601.14228

[96] Hübotter, J., Lübeck, F., Behric, L., Baumann, A., Bagatella, M., Marta, D., Hakimi, I., Shenfeld, I., Buening, T.K., Guestrin, C., Krause, A.: Reinforcement Learning via Self-Distillation. ArXiv (2026). https://arxiv.org/abs/2601.20802

[97] Li, M., Wang, R., Tan, R., Wen, Y.: DCoPilot: Generative AI-Empowered Policy Adaptation for Dynamic Data Center Operations. ArXiv (2026). https://arxiv.org/abs/2602.02137

[98] Khan, R.M.S., Liu, Z., Tan, Z., Fleming, C., Chen, T.: TMS: Trajectory-Mixed Supervision for Reward-Free, On-Policy SFT. ArXiv (2026). https://arxiv.org/abs/2602.03073

[99] Xu, Q., Habib, G., Wu, F., Du, Y., Chen, Z., Mishra, S., Perera, D., Feng, M.: medR: Reward Engineering for Clinical Offline Reinforcement Learning via Tri-Drive Potential Functions. ArXiv (2026). https://arxiv.org/abs/2602.03305

[100] Amer-Yahia, S.: Intelligent Agents for Data Exploration. Proceedings of the VLDB Endowment (2024). https://doi.org/10.14778/3685800.3685913 . https://doi.org/10.14778/3685800.3685913

[101] Furuta, H., Lee, K.-H., Nachum, O., Matsuo, Y., Faust, A., Gu, S.S., Gur, I.: Multimodal Web Navigation with Instruction-Finetuned Foundation Models. ArXiv (2023). https://arxiv.org/abs/2305.11854

[102] Qi, Z., Liu, X., Iong, I.L., Lai, H., Sun, X., Zhao, W., Yang, Y., Yang, X., Sun, J., Yao, S., Zhang, T., Xu, W., Tang, J., Dong, Y.: WebRL: Training LLM Web Agents via Self-Evolving Online Curriculum Reinforcement Learning. ArXiv (2024). https://arxiv.org/abs/2411.02337

[103] Zhuang, Y., Jin, D., Chen, J., Shi, W., Wang, H., Zhang, C.: WorkForceAgent-R1: Incentivizing Reasoning Capability in LLM-based Web Agents via Reinforcement Learning. ArXiv (2025). https://arxiv.org/abs/2505.22942

[104] Liu, J., Li, Y., Zhang, C., Li, J., Chen, A., Ji, K., Cheng, W., Wu, Z., Du, C., Xu, Q., Song, J., Zhu, Z., Chen, W., Zhao, P., He, J.: WebExplorer: Explore and Evolve for Training Long-Horizon Web Agents. ArXiv (2025). https://arxiv.org/abs/2509.06501

[105] Zhang, Y., Zeng, Y., Li, Q., Hu, Z., Han, K., Zuo, W.: Tool-R1: Sample-Efficient Reinforcement Learning for Agentic Tool Use. ArXiv (2025). https://arxiv.org/abs/2509.12867

[106] Tan, W., Qu, X., Tu, M., Ge, M., Liu, A.T., Koehn, P., Lu, L.: Process-Supervised Reinforcement Learning for Interactive Multimodal Tool-Use Agents. ArXiv (2025). https://arxiv.org/abs/2509.14480

[107] Choi, C., Song, H., Kim, D., Jung, W., Cho, M., Park, S., Bae, N., Yu, S., Lim, K.: MENTOR: A Reinforcement Learning Framework for Enabling Tool Use in Small Models via Teacher-Optimized Rewards. ArXiv (2025). https://arxiv.org/abs/2510.18383

[108] Wu, J., Zhao, Q., Chen, Z., Qin, K., Zhao, Y., Wang, X., Yao, Y.: GAP: Graph-Based Agent Planning with Parallel Tool Use and Reinforcement Learning. ArXiv (2025). https://arxiv.org/abs/2510.25320

[109] Jiang, Y., Ferraro, F.: SCRIBE: Structured Mid-Level Supervision for Tool-Using Language Models. arXiv (Cornell University) (2026). https://arxiv.org/abs/2601.03555

[110] Gao, X., Huang, P., Liu, Z., Yan, Y., Wang, S., Chen, Z., Qian, C., Yu, G., Gu, Y.: Teaching LLMs to Learn Tool Trialing and Execution through Environment Interaction. ArXiv (2026). https://arxiv.org/abs/2601.12762

[111] Lin, M., Dai, E., Liu, H., Tang, X., Yan, Y., Dai, Z., Zeng, J., Zhang, Z., Wang, F., Gao, H., Luo, C., Zhang, X., He, Q., Wang, S.: How Far Are LLMs from Professional Poker Players? Revisiting Game-Theoretic Reasoning with Agentic Tool Use. ArXiv (2026). https://arxiv.org/abs/2602.00528

[112] Guo, Y., Yang, W., Yang, S., Liu, Z., Chen, C., Wei, Y., Hu, Y., Huang, Y., Hao, G., Yuan, D., Wang, J., Chen, X., Yu, H., Lei, L., Di, P.: OpAgent: Operator Agent for Web Navigation. ArXiv (2026). https://arxiv.org/abs/2602.13559

[113] Shen, J.-H., Jain, A., Xiao, Z., Amlekar, I., Hadji, M., Podolny, A., Talwalkar, A.: ScribeAgent: Towards Specialized Web Agents Using Production-Scale Workflow Data. Unknown venue (2024). https://doi.org/10.32388/8vog0o . https://doi.org/10.32388/8vog0o

[114] Kannan, S.S., Venkatesh, V.L.N., Min, B.-C.: SMART-LLM: Smart Multi-Agent Robot Task Planning using Large Language Models. ArXiv (2023). https://arxiv.org/abs/2309.10062

[115] Hu, M., Zhou, Y., Fan, W., Nie, Y., Xia, B., Sun, T., Ye, Z., Jin, Z., Li, Y., Chen, Q., Zhang, Z., Wang, Y., Ye, Q., Ghanem, B., Luo, P., Li, G.: OWL: Optimized Workforce Learning for General Multi-Agent Assistance in Real-World Task Automation. ArXiv (2025). https://arxiv.org/abs/2505.23885

[116] Li, W., Lin, J., Jiang, Z., Cao, J., Liu, X., Zhang, J., Huang, Z., Chen, Q., Sun, W., Wang, Q., Lu, H., Qin, T., Zhu, C., Yao, Y., Fan, S., Li, X., Wang, T., Liu, P., Zhu, K., Zhu, H., Shi, D., Wang, P., Guan, Y., Tang, X., Liu, M., Jiang, Y.E., Yang, J., Liu, J., Zhang, G., Zhou, W.: Chain-of-Agents: End-to-End Agent Foundation Models via Multi-Agent Distillation and Agentic RL. ArXiv (2025). https://arxiv.org/abs/2508.13167

[117] Zhou, Y., Zhang, M., Li, K., Wang, M., Liu, Q., Wang, Q., Liu, J., Liu, F., Li, S., Li, W., Gao, M., Kumar, A., Fan, X., Zhao, Z., Zhang, L.: Mixture-of-Minds: Multi-Agent Reinforcement Learning for Table Understanding. ArXiv (2025). https://arxiv.org/abs/2510.20176

[118] Liu, S., Du, D., Yang, T., Li, Y., Qiu, B.: MarsRL: Advancing Multi-Agent Reasoning System via Reinforcement Learning with Agentic Pipeline Parallelism. ArXiv (2025). https://arxiv.org/abs/2511.11373

[119] Sharma, A., Arafat, S.Y., Sharma, J.K., Huang, K.: Bilevel Optimization for Covert Memory Tampering in Heterogeneous Multi-Agent Architectures (XAMT). ArXiv (2025). https://arxiv.org/abs/2512.15790

[120] Zhou, Z., Liu, Z., Liu, J., Shao, Q., Wang, Y., Shao, K., Jin, D., Xu, F.: ResMAS: Resilience Optimization in LLM-based Multi-agent Systems. ArXiv (2026). https://arxiv.org/abs/2601.04694

[121] Zou, M., Chen, J., Luo, A., Dai, J., Zhang, C., Sun, D., Xu, Z.: FinEvo: From Isolated Backtests to Ecological Market Games for Multi-Agent Financial Strategy Evolution. ArXiv (2026). https://arxiv.org/abs/2602.00948

[122] Xu, Z., Xu, Z., Zhang, R., Zhu, C., Yu, S., Liu, W., Zhang, Q., Ding, W., Yu,

C., Wang, Y.: WideSeek-R1: Exploring Width Scaling for Broad Information Seeking via Multi-Agent Reinforcement Learning. ArXiv (2026). https://arxiv.org/abs/2602.04634

[123] Despature, J.Y., Shibata, K., Matsubara, T.: CoLF: Learning Consistent Leader-Follower Policies for Vision-Language-Guided Multi-Robot Cooperative Transport. ArXiv (2026). https://arxiv.org/abs/2602.07776

[124] Li, H., Yu, C., Stepputtis, S., Campbell, J., Hughes, D., Lewis, C., Sycara, K.: Theory of Mind for Multi-Agent Collaboration via Large Language Models. Unknown venue (2023). https://doi.org/10.18653/v1/2023.emnlp-main.13 . https://doi.org/10.18653/v1/2023.emnlp-main.13

[125] Artificial Intelligence 2026, A., Ma, Q., Xie, X., Xue, X., Zhang, X.: Adaptive Regulation via Dual-Layer Evolution (ARDE): A Multi-Agent Approach to Balancing Efficiency, Fairness, and Diversity in Crowdsourced Platforms. Underline Science Inc. (2026). https://doi.org/10.48448/8p81-ba98 . https://doi.org/10.48448/8p81-ba98